

YASSINE CHETTOUCH
VERTO · ENGINEERING

ENGINEERING · JANUARY 2026
TECHNICAL HANDOVER

Verto Engineering Handbook

ENG

PART I
Engine

PART II
Product

PART III
Runtime

APPENDIX
Services

Contents.

Chapters	Page
Architecture overview	003
The engine	
Identity & security	008
Financial ledger	022
Intelligence pipeline	039
The experience	
User workspace	067
Governance & operations	087
Design engine & brand	094
Infrastructure	
Infrastructure & orchestration	103
Technical dependencies	110
Job runbook	062

Figures	Page
01 High-level topology	004
02 Authentication state machine	010
03 Middleware sequence	015
04 Ledger entity relationships	024
05 Financial data flow	031
06 Vision architecture	041
07 Pipeline lifecycle	043
08 Contextual batching	047
09 Editor composition	070
10 Regeneration loop	075
11 Realtime synchronization	081
12 Admin action sequence	089
Architecture decisions	064

ARCHITECTURE

Architecture overview

MAP

A map of the seven systems and their implementation boundaries.

IN THIS SECTION

01 System topology P. 004

02 Seven systems P. 005

03 Technology matrix P. 006

04 Codebase map P. 006

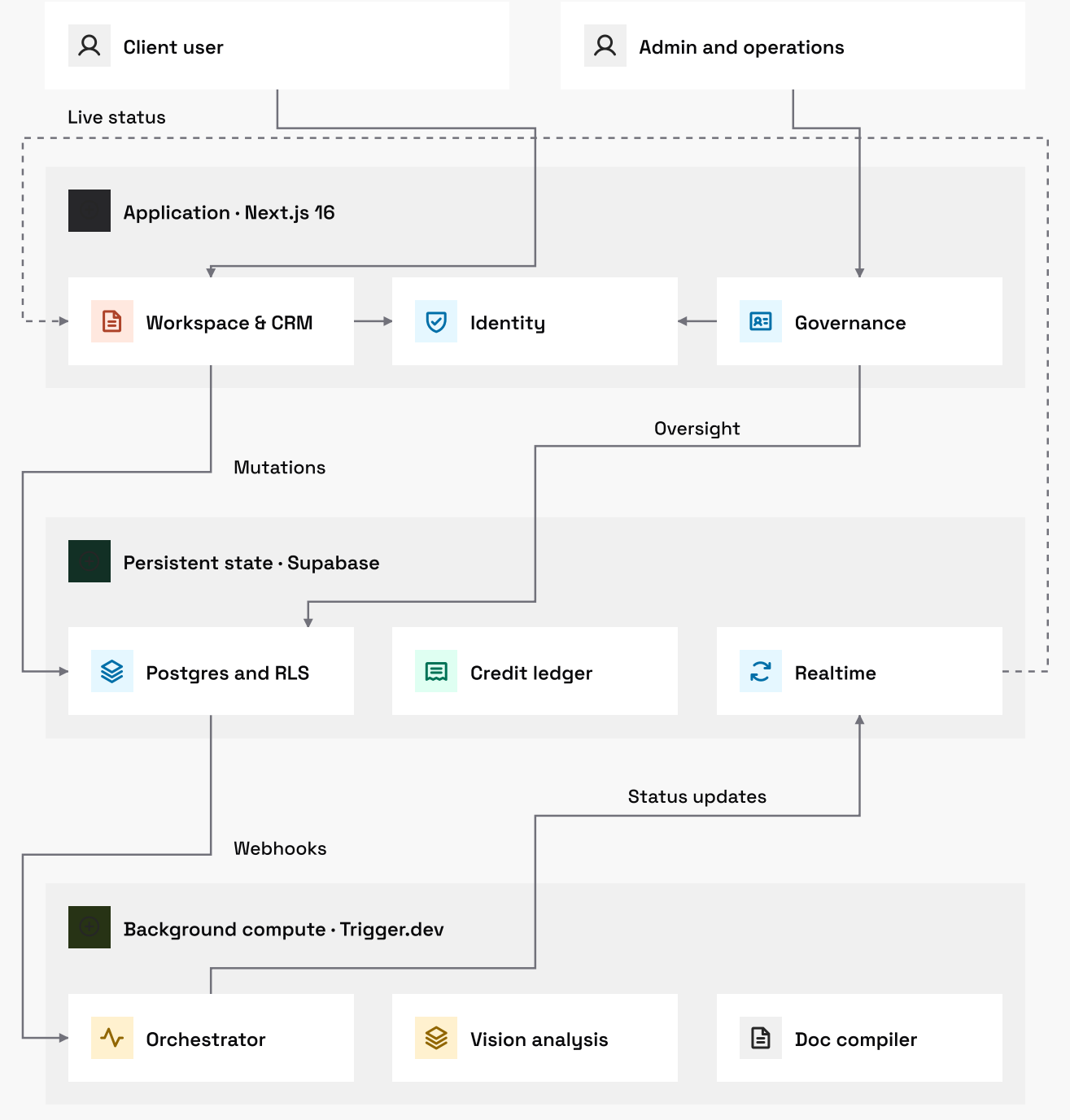
05 Developer quickstart P. 007

Verto translates page images into an editable document tree that represents inferred text and structure. A reviewer can correct that tree before the Google Docs mapper generates native document operations and exports.

This handbook explains seven systems and their implementation boundaries. Listings are partial source windows; read the complete referenced files. Live providers and deployments require separate verification. Publication license: CC BY-NC-ND 4.0. Embedded fonts: SIL Open Font License.

1. System topology

01 High-level topology



2. Seven systems

Use the system map to find the owner of a concern. Each chapter then moves from responsibility to implementation, failure handling, and operational reference.

01 **Identity & security**

Authentication, MFA, sessions, and tenant data boundaries.

`src/actions/auth` · `supabase/migrations`

02 **Financial ledger**

Credit purchases, deductions, refunds, and balance history.

`src/actions/billing` · `src/utils/supabase/credits.ts`

03 **Intelligence pipeline**

Page images, translation, quality checks, and export.

`trigger` · `src/utils/ai` · `src/utils/gcp`

04 **User workspace**

Document editing, source review, client records, and live state.

`src/components/translations/translation-editor` · `src/actions/services`

05 **Governance & operations**

Privileged actions, user recovery, credit adjustments, and reporting.

`src/app/(protected)/admin` · `src/actions/admin`

06 **Design engine & brand**

Product components, semantic tokens, notifications, and assets.

`src/components/ui` · `src/app/globals.css`

07 **Infrastructure & orchestration**

Hosting, persistent state, workers, networking, and deployment.

`src/env.ts` · `trigger.config.ts`

3. Technology matrix

LAYER	CORE TECHNOLOGY	SUPPORTING TOOLS
Frontend	React and Next.js	Radix UI · Motion · Phosphor · Tailwind CSS
State	Server Actions	TanStack Query · Zustand
Editor	TipTap	ProseMirror document model
Background work	Trigger.dev	Supabase client and task checkpoints
AI adapters	OpenAI Responses API	Models in `src/config/ai.ts`
Documents	Google Docs	Google Docs and Drive APIs

4. Codebase map

The codebase groups implementation by domain, with shared infrastructure beneath it.

REPOSITORY MAP LISTING LINES 1–19

```
01 src/
02 ├── actions/              Server actions and authorization
03 ├── app/                  Routes and request handlers
04 ├── components/           Application and editor UI
05 ├── config/               Model and service configuration
06 ├── engine/
07 │   ├── document/         Shared tree, transforms and validation
08 │   └── translation/      Isolated translation primitives
09 ├── hooks/                Client state and queries
10 └── utils/
11     ├── ai/               Model adapters and vision processing
12     ├── gcp/              Google Docs and Drive mapping
13     └── supabase/         Typed database clients and ledger helpers
14 trigger/                  Durable processing tasks
15 supabase/migrations/      One current-schema baseline
16 tests/                    Offline characterization and boundary tests
17 fixtures/                 Synthetic multi-page, RTL and Japanese inputs
18 docs/                     Setup, configuration, architecture and ADRs
19 handbook/                 This publication and its license
```

5. Developer quickstart

Use inert configuration and the synthetic editor preview for local evaluation. Connected services require accounts and credentials under your control; see `docs/setup.md`.

PREREQUISITES

Runtime

Node and pnpm versions are pinned in `.node-version` and `package.json`.

Local evaluation

The offline preview uses local fixtures and blocks external Node connections.

Connected services

Supabase, Trigger.dev, OpenAI, Google, Stripe and Resend require explicit configuration.

BOOT SEQUENCE

README.MD · OFFLINE EVALUATION LISTING LINES 1–7

```
01 node --version
02 npm install --global pnpm@10.22.0
03 pnpm install --frozen-lockfile
04 pnpm check
05 pnpm build:offline
06 pnpm start:offline
07 # Open http://localhost:3000/preview
```

ESSENTIAL COMMANDS

Verify

`pnpm check` runs formatting, lint, strict types and offline transform coverage.

Build and preview

`pnpm build:offline`, then `pnpm start:offline`; open `/preview`.

Inspect decisions

Read `docs/architecture.md` and the numbered files in `docs/decisions/`.

PART I · THE ENGINE

Identity & security

01

Authentication, MFA, sessions, and tenant isolation.

IN THIS SECTION

01 Identity boundaries P. 009

02 Authentication flow P. 010

03 Multi-factor authentication P. 013

04 Session middleware P. 015

05 Tenant isolation P. 016

06 Identity lifecycle P. 018

07 Security controls and excerpt limits P. 021

Modules	`src/actions/auth/*` · `supabase/migrations/*`

1. Identity boundaries

Authentication separates identity from permission to use a protected route or action. A valid session is only the starting point; tenant, account-status, and assurance checks determine access.

Supabase Auth provides the identity and session primitives. Verto's server-side controls require a second factor where the application's MFA policy demands it.

ACCESS CHECKS

Session identity

Supabase Auth supplies identity and session primitives. A session is the starting point for access checks.

MFA assurance

Sensitive routes and actions must enforce the required assurance level. Password sign-in alone can remain at AAL1.

Data ownership

RLS policies bind ordinary translation access to the owning profile. Administrative access needs a separate authorized path.

Sensitive changes

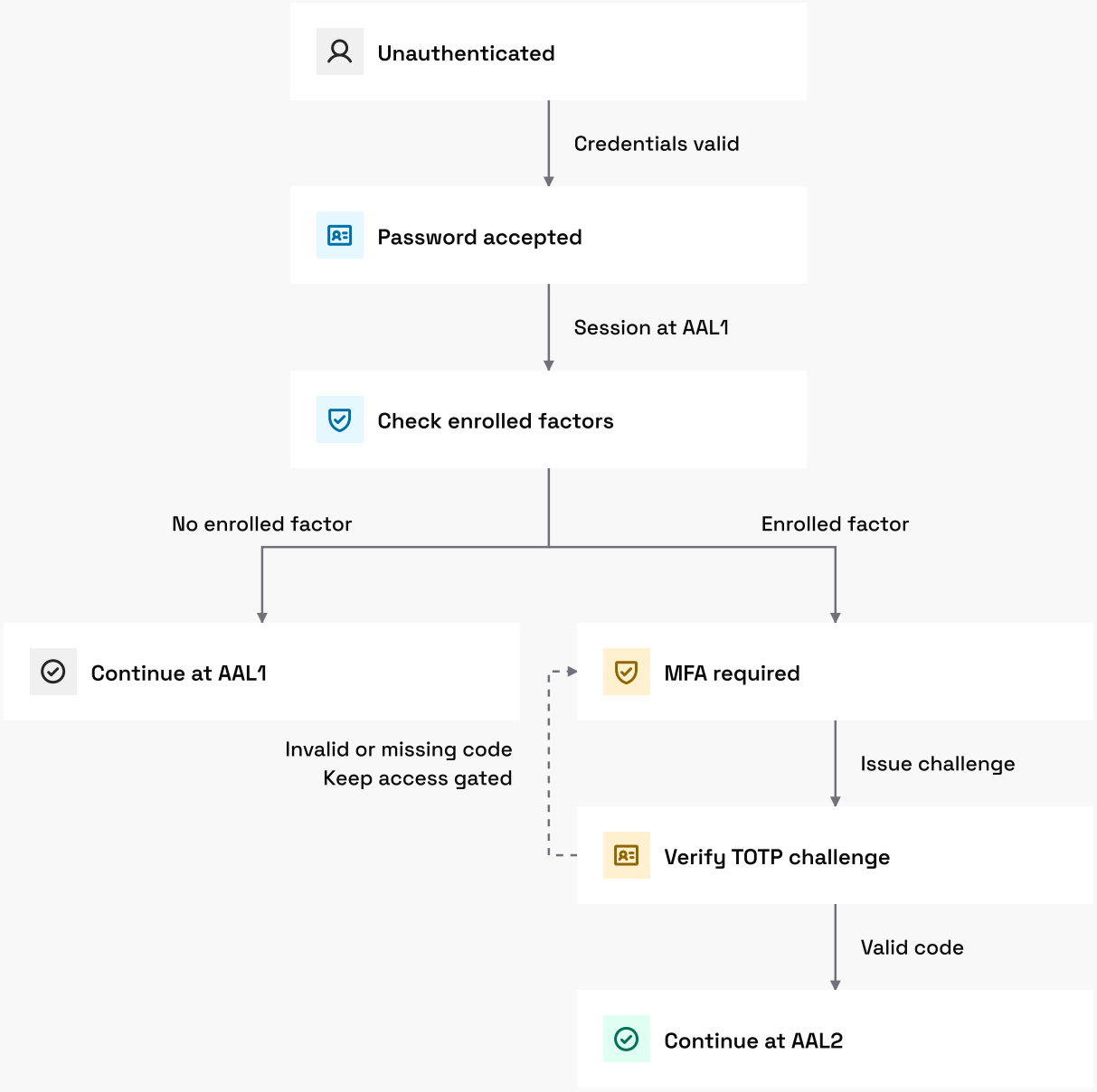
Password changes and account deletion use fresh password verification in the documented flows.

2. Authentication flow

Implementation `src/actions/auth/sign-in.ts`

The password sign-in action checks credentials and enrolled factors. The request proxy and user-validation helper check assurance for protected access; inspect their full branches alongside the excerpt.

02 **Authentication state machine**



Password sign-in and access assurance are separate: AAL1 does not satisfy an AAL2 gate.

2.1 Password sign-in and MFA gate

Password sign-in may establish an `aal1` session. When MFA is required, the application must keep sensitive routes and actions unavailable until the session reaches `aal2`.

01 Submit credentials

Sign in with the user's email and password.

02 Inspect assurance

Check the enrolled factors and the session's current and required authentication assurance levels.

03 Require verification

When the session requires `aal2`, route the user to the challenge and block protected actions until the second factor is verified.

SRC/ACTIONS/AUTH/SIGN-IN.TS · L60–83 LISTING LINES 1–24

```
const { data, error } = await supabase.auth.signInWithPassword({
    email: email,
    password: password,
});

if (error) {
    const resolvedError = standardizeError('INVALID_CREDENTIALS');
    logger.error(['Failed to sign in with password:', error]);
    return resolvedError;
}

const { session, user } = data;

if (session) {
    logger.info(['User has successfully logged in:', { user }]);
}

if (user?.id && !user?.confirmed_at) {
    return {
        code: 'EMAIL_NOT_CONFIRMED',
        description: 'We have sent you an email to confirm your email address.',
        message: 'Your email is not confirmed!',
        status: 401,
        title: 'Email Not Confirmed',
```

Assurance boundary

Password acceptance and MFA completion are separate. Follow the assurance checks in `src/utils/supabase/middleware.ts` and `src/helpers/validate-user.ts`.

2.2 Registration and profile creation

Implementation `src/actions/auth/sign-up.ts`

Registration supplies metadata to Supabase Auth. The baseline's `handle_new_user` trigger creates the public profile, balance and ordinary user role. User-supplied metadata does not grant the administrative role.

SRC/ACTIONS/AUTH/SIGN-UP.TS · L28–45 LISTING LINES 1–18

```
const registerUser = async (supabase: SupabaseClient, validFields: z.infer<typeof RegisterSchema>) => {
    const { data, error } = await supabase.auth.signUp({
        email: validFields.contactInformation.email,
        options: {
            data: {
                address: validFields.contactInformation.address,
                avatar_url: generateGravatarUrl(validFields.contactInformation.email),
                city: validFields.contactInformation.city,
                company_name: validFields.professionalInformation.companyName,
                country: validFields.contactInformation.country,
                date_of_birth: validFields.personalInformation.dateOfBirth,
                first_name: validFields.personalInformation.firstName,
                is_profile_complete: true,
                last_name: validFields.personalInformation.lastName,
                phone: validFields.contactInformation.phoneNumber,
                professional_affiliations: validFields.professionalInformation.professionalAffiliations,
                qualifications: validFields.professionalInformation.qualifications,
                role: 'user',
```

2.3 Authentication callback

Implementation `src/app/auth/callback/route.ts`

The server-side callback exchanges an OAuth or passwordless sign-in code for a session, then completes the application's callback handling.

Sign-in event

The callback handler records `sign_in` in `user_activity`, including callback-based social sign-ins. Check the insert result and retention policy when auditing activity coverage.

SRC/APP/AUTH/CALLBACK/ROUTE.TS · L10-29 LISTING LINES 1-20

```ts
export async function GET(request: Request) {
    const logger = getLogger('AUTH:CALLBACK');

    const { origin, searchParams } = new URL(request.url);
    const code = searchParams.get('code');
    const next = searchParams.get('next') ?? '/';

    if (code) {
        const supabase = await createServerClient();
        const {
            data: { session, user },
            error,
        } = await supabase.auth.exchangeCodeForSession(code);

        if (!error) {
            const isLocalEnv = process.env.NODE_ENV === 'development';
            let redirectUrl;

            if (isLocalEnv) {
                redirectUrl = new URL(`${env.NEXT_PUBLIC_APP_URL}${next}`);
```

3. Multi-factor authentication

Implementation `src/actions/auth/multi-factor-authentication.ts`

The documented second factor uses TOTP authenticator codes. Enrollment and verification raise session assurance from `aal1` to `aal2`; sensitive actions must check that assurance server-side.

3.1 Challenge and verification

Server actions submit MFA challenges and verify the response with the identity provider.

01 Fetch factors

Retrieve the `factorId` for the user's registered TOTP device.

02 Issue challenge

Call `supabase.auth.mfa.challenge({ factorId })`.

03 Verify code

Verify the six-digit code with `verifyChallenge`. A valid response upgrades the session from AAL1 (password) to AAL2 (MFA).

SRC/ACTIONS/AUTH/MULTI-FACTOR-AUTHENTICATION.TS · L20–29 LISTING LINES 1–10

```
export async function createAndVerifyMfaChallenge(
    factorId: string,
    code: string,
): Promise<
    ApiResponseOrError<{
        access_token: string;
        token_type: string;
        expires_in: number;
        refresh_token: string;
        user: User;
```

3.2 Sign-out and local cleanup

Implementation `src/actions/auth/sign-out.ts`

The sign-out action calls Supabase Auth sign-out, then clears and resets its query client. Browser cache isolation and navigation require review alongside the calling UI.

SRC/ACTIONS/AUTH/SIGN-OUT.TS · L14–23 LISTING LINES 1–10

```
export const signOut: () => Promise<ApiResponseOrError> = async () => {
    const logger = getLogger('ACTIONS:AUTH:SIGN_OUT');
    try {
        const supabase = await createServerClient();
        const queryClient = getQueryClient();

        const {
            data: { user },
            error: userError,
        } = await supabase.auth.getUser();
```

3.3 Factor enrollment

Enrollment requires a successful verification code before the factor is treated as ready for use.

Enroll

The server action calls Supabase Auth MFA enrollment and returns its QR-code and secret data to the enrollment flow.

Confirm

The application submits a challenge and verification code. Inspect the provider result and assurance level before allowing protected access.

4. Session middleware

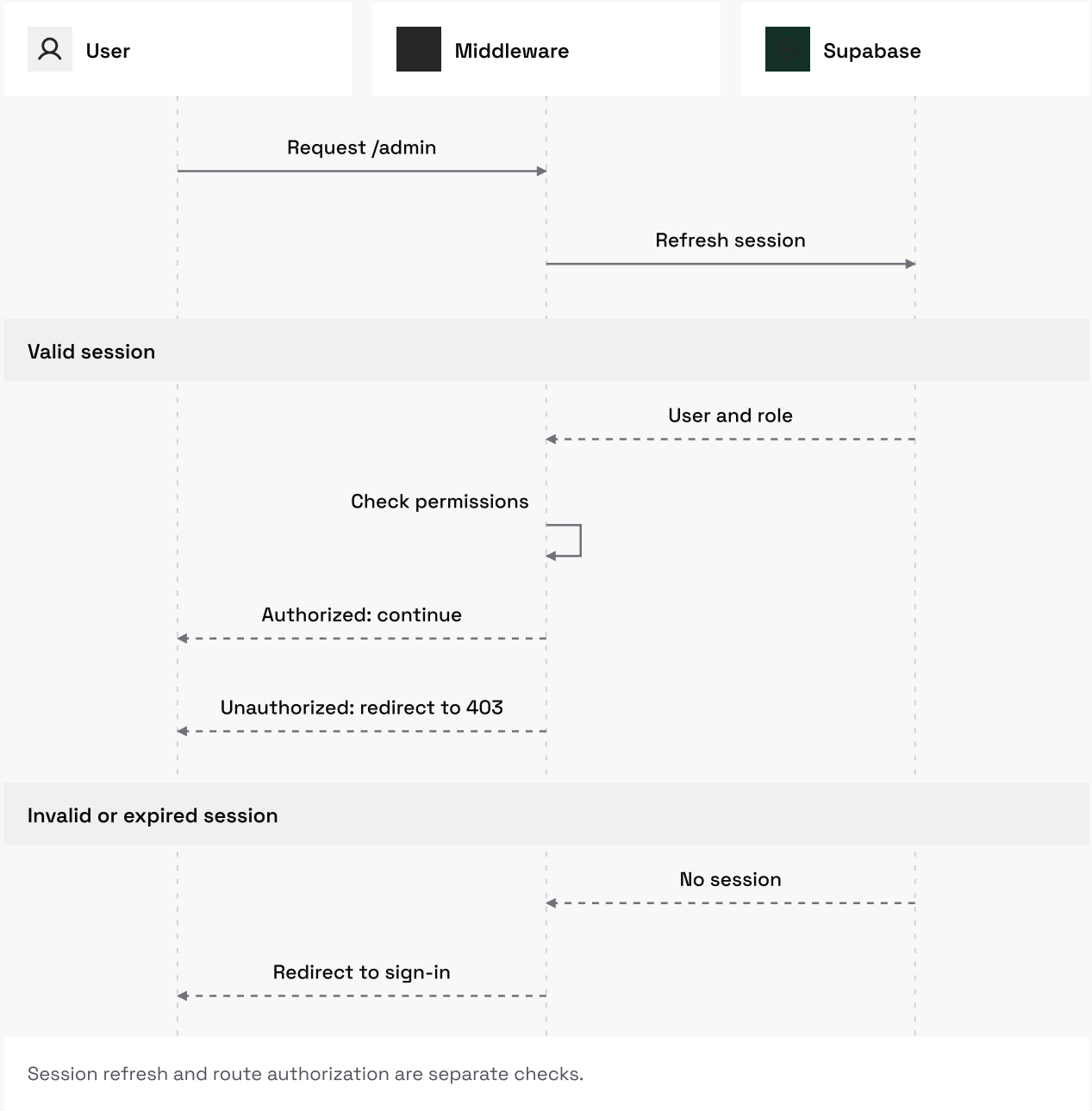
Implementation `src/utils/supabase/middleware.ts`

`updateSession` refreshes authentication state and applies the middleware's route checks. Protected actions and database policies still need their own authorization boundaries.

4.1 Refresh before authorization

The middleware attempts to refresh authentication state before applying its route checks.

03 Middleware sequence



Session refresh and route authorization are separate checks.

4.2 Route and assurance checks

Implementation `src/utils/supabase/middleware.ts`

SRC/UTILS/SUPABASE/MIDDLEWARE.TS · L39–69 LISTING LINES 1–31

```
export async function updateSession(request: NextRequest) {
    const logger = getLogger('MIDDLEWARE:UPDATE_SESSION');
    const shouldDebug = process.env.NODE_ENV === 'development';

    // Generate nonce and CSP header:
    const nonce = Buffer.from(crypto.randomUUID()).toString('base64');
    const cspHeader = `
        default-src 'self' https://accounts.google.com/gsi/;
        script-src 'self' 'nonce-${nonce}' 'strict-dynamic' https://accounts.google.com/gsi/client;
        style-src 'self' 'nonce-${nonce}' https://accounts.google.com/gsi/style;
        img-src 'self' blob: data:;
        font-src 'self';
        object-src 'none';
        base-uri 'self';
        form-action 'self';
        frame-ancestors 'none';
        upgrade-insecure-requests;
        connect-src https://accounts.google.com/gsi/;
        frame-src https://accounts.google.com/gsi/;
    `
        .replace(/\s{2,}/g, ' ')
        .trim();

    // Set request headers:
    const requestHeaders = new Headers(request.headers);
    requestHeaders.set('x-nonce', nonce);
    requestHeaders.set('Content-Security-Policy', cspHeader);

    // Add Next.js Pathname to headers, for use in the layout component:
    const headersPathname = request.nextUrl.pathname;
    requestHeaders.set('next-url', headersPathname);
```

5. Tenant isolation

Implementation `supabase/migrations/00000000000000_baseline.sql`

The documented access model enables RLS on application tables and permits access through explicit policies. Review the deployed policies and grants to establish the actual boundary.

5.1 Tenant access policy

A tenant must not read another tenant's documents. The translation policy binds ordinary user access to the owning profile.

Translation policy

SUPABASE/MIGRATIONS/00000000000000_BASELINE.SQL · L718-722 LISTING LINES 1-5

```
CREATE POLICY "User Manage Own Customers" ON public.customers FOR ALL USING (auth.uid() = profile_id) WITH CHECK (auth.uid() = profile_id);
CREATE POLICY "User Manage Own Translations" ON public.translations FOR ALL USING (auth.uid() = profile_id) WITH CHECK (auth.uid() = profile_id);
CREATE POLICY "User View Own Exports" ON public.translation_exports FOR SELECT USING (EXISTS (SELECT 1 FROM public.translations t WHERE t.id = translation_id AND t.profile_id = auth.uid()));
CREATE POLICY "User Read Own Credits" ON public.credits FOR SELECT USING (auth.uid() = profile_id);
CREATE POLICY "User Read Own Transactions" ON public.credits_transactions FOR SELECT USING (auth.uid() = profile_id);
```

5.2 Administrative access

Administrative access requires a separate authorized policy or an elevated server client. The excerpt below shows the policy-based exception:

SUPABASE/MIGRATIONS/00000000000000_BASELINE.SQL · L382-395 LISTING LINES 1-14

```
CREATE OR REPLACE FUNCTION public.is_admin() RETURNS BOOLEAN AS $$
DECLARE
  user_role public.app_role;
BEGIN

  IF (auth.jwt() ->> 'user_role')::public.app_role = 'ADMIN'::public.app_role THEN
    RETURN TRUE;
  END IF;

  IF EXISTS (
    SELECT 1 FROM public.user_roles
    WHERE user_id = auth.uid()
    AND role = 'ADMIN'::public.app_role
  ) THEN
```

5.3 Financial log permissions

Ordinary users can read their own credit transactions. Administrative policies and the server role have broader access, so the ledger is not immutable against privileged operators.

Ordinary access

Read policies expose owned entries. No ordinary user policy permits transaction update or deletion.

Privileged access

Administrators and the service role can alter data. Operational controls must govern that access.

6. Identity lifecycle

Identity operations cover recovery, account deletion, sensitive changes, and email confirmation.

6.1 Password recovery

Implementation `src/actions/auth/forgot-password.ts`

Password recovery sends a Supabase recovery link back to the application. Use a configured, allowlisted application origin for that redirect; a request's `Origin` header alone is not a trust boundary.

SRC/ACTIONS/AUTH/FORGOT-PASSWORD.TS · L21–29 LISTING LINES 1–9

```
export async function forgotPassword(email: string): Promise<ApiResponseOrError> {
    const logger = getLogger('ACTIONS:AUTH:FORGOT_PASSWORD');
    try {
        const supabase = await createServerClient();
        const origin = (await headers()).get('origin');

        const { data, error } = await supabase.auth.resetPasswordForEmail(email, {
            redirectTo: `${origin}/auth/reset-password`,
        });
```

6.2 Account deletion

Implementation `src/actions/auth/close-user-account.ts`

Account deletion calls `verify_user_password`, checks RPC errors and its error field, then invokes the server-only administrative delete API for the authenticated user.

SRC/ACTIONS/AUTH/CLOSE-USER-ACCOUNT.TS · L13–25 LISTING LINES 1–13

```
01 export async function closeUserAccount({ password }: { password: string }):
   Promise<ApiResponseOrError> {
02     const logger = getLogger('ACTIONS:AUTH:CLOSE_ACCOUNT');
03
04     try {
05         await validateUser();
06
07         // Create a connection to the Supabase server.
08         const supabase = await createServerClient();
09
10         const { user } = await validateUser();
11
12         if (!password) {
13             const errorResponse = standardizeError('PASSWORD_INVALID');
```

6.3 High-assurance actions

Implementation `src/actions/auth/change-user-password.ts`

Password-change and account-deletion actions verify the supplied password with the RPC before continuing. Session, assurance and tenant checks remain separate controls.

SRC/ACTIONS/AUTH/CHANGE-USER-PASSWORD.TS · L23–31 LISTING LINES 1–9

```
01 export async function changeUserPassword(
02     currentPassword: string,
03     newPassword: string,
04 ): Promise<ApiResponseOrError> {
05     const logger = getLogger('ACTIONS:AUTH:CHANGE_PASSWORD');
06     try {
07         await validateUser();
08
09         const supabase = await createServerClient();
```

6.4 Email change confirmation

Implementation `src/actions/auth/change-user-email.ts`

Email changes require confirmation. The source uses Next.js `after()` for deferred activity logging; check its completion behavior where an audit record is required.

SRC/ACTIONS/AUTH/CHANGE-USER-EMAIL.TS · L15–25 LISTING LINES 1–11

```
01 export async function changeUserEmail(email: string): Promise<ApiResponseOrError> {
02     const logger = getLogger('ACTIONS:AUTH:CHANGE_EMAIL');
03     try {
04         const { user } = await validateUser();
05
06         logger.info(`Changing the email address to [${email}] for the current
   authenticated user.`);
07
08         const supabase = await createServerClient();
09
10         const { data, error } = await supabase.auth.updateUser({
11             email,
```

7. Security controls and excerpt limits

CONTROL	IMPLEMENTATION BOUNDARY	SOURCE
Authentication	Password and OAuth adapters	`src/actions/auth/`
MFA	TOTP challenge and verification	`src/actions/auth/multi-factor-authentication.ts`
Session	Supabase SSR cookie adapter; verify deployed cookie attributes	`src/utils/supabase/middleware.ts`
Callback	Authorization-code exchange	`src/app/auth/callback/route.ts`
Data access	Row-level policies and restricted RPC grants	`supabase/migrations/00000000000000_baseline.sql`
Lifecycle	Password verification before sensitive changes	`src/actions/auth/change-user-password.ts`

EXCERPT BOUNDARY

The sign-in excerpt does not demonstrate a session-null check. Verify assurance-level enforcement in middleware, protected actions, and database policies. This handbook documents the design and source examples; it does not certify the deployed security configuration.

PART I · THE ENGINE

Financial ledger

02

Transactions, credit integrity, payments, and refunds.

IN THIS SECTION

01 Ledger architecture P. 023
02 Ledger schema P. 023
03 Billing interface P. 025
04 Concurrent credit deduction P. 026
05 Payment ingestion P. 028
06 Failure refunds P. 029
07 Financial data flow P. 031
08 Pricing rules P. 032
09 Manual adjustments P. 032
10 Financial reporting P. 033
11 Balance refresh and checkout P. 034
12 Integrity controls P. 036
13 Operational configuration P. 037

Modules	`src/actions/billing/*` · `src/utils/supabase/credits.ts` · `supabase/migrations/00000000000000_baseline.sql`

1. Ledger architecture

The financial system uses a transaction ledger. Purchases, consumption, and refunds are recorded as events, with a cached balance maintained by a database trigger.

Typed ledger events record credit purchases, consumption, and refunds. The cached balance is a projection of that history, rather than an independent application-side counter.

The `sync_credits` trigger updates the cached balance when ledger entries are inserted. Its consistency depends on the trigger, permissions, transaction boundaries, and the set of transaction types it recognizes.

2. Ledger schema

Implementation: `supabase/migrations/00000000000000_baseline.sql`.

`credits_transactions` stores the history of balance-changing events.

2.1 Tables and responsibilities

SUPABASE/MIGRATIONS/00000000000000_BASELINE.SQL · L295-318 LISTING LINES 1–24

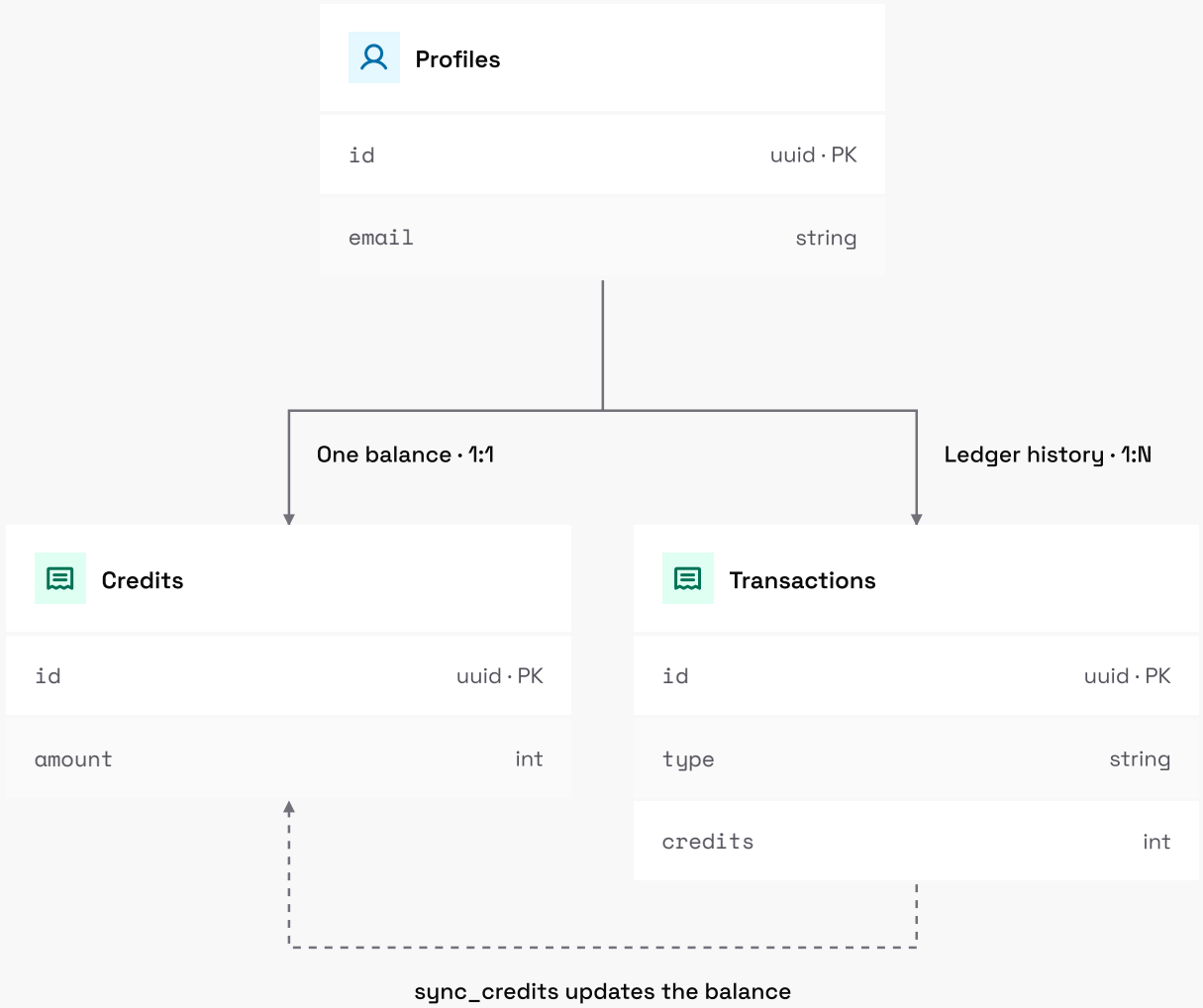
```
CREATE TABLE public.credits (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    profile_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE UNIQUE,
    amount INTEGER NOT NULL DEFAULT 0 CHECK (amount >= 0),
    lifetime_purchased INTEGER NOT NULL DEFAULT 0,
    lifetime_used INTEGER NOT NULL DEFAULT 0,
    last_purchase_at TIMESTAMPTZ,
    last_usage_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TRIGGER set_timestamps_on_credits
    BEFORE INSERT OR UPDATE ON public.credits
    FOR EACH ROW EXECUTE FUNCTION public.set_timestamps();

CREATE TABLE public.credits_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    profile_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
    translation_id UUID REFERENCES public.translations(id) ON DELETE SET NULL,
    credits INTEGER NOT NULL CHECK (credits > 0),
    type VARCHAR(50) NOT NULL,
    description TEXT,
    details JSONB DEFAULT '{}'::jsonb,
```

2.2 Entity relationships

04 Ledger entity relationships



amount is the current balance; credits is the transaction delta.
The source ERD lists purchase and consumption transaction types.

2.3 Balance synchronization trigger

The documented write path inserts into `credits_transactions`. The trigger below applies recognized transaction types to `credits.amount`.

SUPABASE/MIGRATIONS/00000000000000_BASELINE.SQL · L446-455 LISTING LINES 1–10

```
CREATE OR REPLACE FUNCTION sync_credits() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.type IN ('purchase', 'bonus', 'refund') THEN
        UPDATE public.credits SET amount = amount + NEW.credits WHERE profile_id = NEW.profile_id;
    ELSIF NEW.type IN ('penalty', 'consumption') THEN
        UPDATE public.credits SET amount = amount - NEW.credits WHERE profile_id = NEW.profile_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

3. Billing interface

Implementation `src/app/(protected)/account/*`

Two account views expose the ledger: credit purchase and billing history.

3.1 Credit purchase

Implementation `src/app/(protected)/account/credits/top-up/top-up-page.tsx`

The top-up view estimates the purchase and opens checkout:

Purchase calculation

`src/constants/credits.ts` defines unit pricing, volume discounts and preset totals. These are application settings, not a current service offer.

Checkout

The server adapter creates Stripe Checkout Sessions; completion is handled through the signed webhook.

Balance

Query refreshes retrieve the database projection after billing actions.

3.2 Customer billing statement

Implementation `src/app/(protected)/account/billing`

The billing view presents transaction history and payment-document links.

Invoice history	Renders the `credits_transactions` history.
PDF receipts	Links directly to Stripe-hosted invoice PDFs.

3.3 Transaction history

The history view composes `TransactionHistoryPanel`, `CreditsTransactionTable`, and `FilterTransactionsForm`.

The transaction history provides pagination and filters for users reviewing a large ledger.

Infinite scroll

Load transaction history with `useInfiniteQuery` and an `IntersectionObserver` as the user scrolls.

Filters

Narrow the history by transaction type, amount, or date.

- **Type:** `consumption`, `purchase`, `bonus`, `penalty`, or `refund`.
- **Amount:** Minimum and maximum credit values.
- **Date:** A range selected in the calendar.

Transaction types

Color distinguishes transaction types, including green purchases and red consumption.

4. Concurrent credit deduction

Implementation `src/utils/supabase/credits.ts`

Concurrent requests must not spend the same balance twice. With 10 credits available, two simultaneous 10-credit requests must be serialized at the database boundary.

The shown PostgreSQL RPC locks the balance row while checking and deducting credits. Keep the deduction and ledger insertion within the same database transaction.

4.1 Deduction sequence

Implementation `deduct_credits_for_translation`

The deduction RPC checks available credits and inserts the consumption event. `sync_credits` applies the balance change inside the transaction.

THE TRANSACTION BLOCK

Authorize

Require the caller's account or service role, positive counts and an owned translation.

Lock and check

Lock the balance with `FOR UPDATE` and compare available credits.

Record and project

Insert consumption; `sync_credits` updates the balance within the transaction.

SUPABASE/MIGRATIONS/00000000000000_BASELINE.SQL · L468-509 LINES 1-25

```
01 CREATE OR REPLACE FUNCTION deduct_credits_for_translation(
02   p_user_id UUID,
03   p_translation_id UUID,
04   p_credits_required INTEGER,
05   p_document_name TEXT,
06   p_page_count INTEGER
07 ) RETURNS JSONB AS $$
08 DECLARE
09   v_current_balance INTEGER;
10   v_transaction_id UUID;
11 BEGIN
12   IF (auth.jwt() ->> 'role') IS DISTINCT FROM 'service_role' AND auth.uid() IS DISTINCT
   FROM p_user_id THEN
13     RAISE EXCEPTION 'Unauthorized: You can only deduct your own credits.';
14   END IF;
15
16   IF p_credits_required IS NULL OR p_page_count IS NULL OR p_credits_required <= 0 OR
   p_page_count <= 0 THEN
17     RAISE EXCEPTION 'Credit and page counts must be positive.';
18   END IF;
19   IF NOT EXISTS (SELECT 1 FROM public.translations WHERE id = p_translation_id AND
   profile_id = p_user_id) THEN
20     RAISE EXCEPTION 'Translation is unavailable for this account.';
21   END IF;
22
23   SELECT amount INTO v_current_balance FROM public.credits WHERE profile_id = p_user_id
   FOR UPDATE;
24
25   IF NOT FOUND THEN RAISE EXCEPTION 'Credits record not found'; END IF;
```

SUPABASE/MIGRATIONS/00000000000000_BASELINE.SQL · L468-509 LINES 26-42

```
  IF v_current_balance < p_credits_required THEN
    RETURN jsonb_build_object('success', false, 'error', 'Insufficient credits');
  END IF;

  INSERT INTO public.credits_transactions (
    profile_id, translation_id, credits, type, description, details
  ) VALUES (
    p_user_id, p_translation_id, p_credits_required, 'consumption',
    'Translation: ' || p_document_name, jsonb_build_object('page_count', p_page_count)
  )
  RETURNING id INTO v_transaction_id;

  RETURN jsonb_build_object('success', true, 'new_balance', v_current_balance -
p_credits_required, 'transaction_id', v_transaction_id);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, auth, extensions,
pg_temp;

CREATE OR REPLACE FUNCTION public.handle_new_user() RETURNS TRIGGER
```

5. Payment ingestion

Implementation

`src/actions/billing/stripe/server-only/events/handle-completed-checkout-session.ts`

The checkout interface starts payment; the webhook handler fulfills the purchase after verifying the provider event.

5.1 Webhook processing

Start checkout

Attach the account and credit information to the Checkout Session.

Verify the event

The webhook adapter verifies the signed provider event. Connected payment-method behavior requires provider tests.

Guard duplicates

The handler checks the session ID; a unique purchase-session index rejects concurrent duplicate inserts.

Record purchase

Insert a purchase entry and check the result. The ledger trigger updates the balance.

5.2 Webhook handler excerpt

SRC/ACTIONS/BILLING/STRIPE/SERVER-ONLY/EVENTS/HANDLE-COMPLETED-CHECKOUT-SESSION.TS · L47–70 LISTING LINES 1–24

```
    .from('credits_transactions')
    .select('id')
    .contains('details', { session_id: session.id })
    .maybeSingle();

if (existingTransaction) {
    return logger.info(['Session already processed, skipping:', session.id]);
}

try {
    const CURRENCY = expandedSession.currency ?? 'usd';

    const intent = expandedSession.payment_intent as Stripe.PaymentIntent | null;
    const invoice = (intent as (Stripe.PaymentIntent & { invoice?: Stripe.Invoice | null }) | null)
        ?.invoice as Stripe.Invoice | null;
    const discountObject = invoice?.discounts?.[0] as Stripe.Discount | null;
    const charge = intent?.latest_charge as Stripe.Charge | null;
    const balanceTransaction = charge?.balance_transaction as Stripe.BalanceTransaction | null;
    const method = intent?.payment_method as Stripe.PaymentMethod | null;
    const creditsPurchased = Number(expandedSession.metadata?.credits ?? 0);

    const discountPercentOff: number =
        (discountObject as (Stripe.Discount & { coupon?: Stripe.Coupon }) | null)?.coupon?.percent_off ||
        parseInt(expandedSession.metadata?.discount as unknown as string) ||
```

DATABASE ENFORCEMENT

The baseline's `credits_transactions_purchase_session_unique` index prevents two purchase entries with the same non-null checkout session. The handler also performs a session lookup. This safeguards duplicate insertion; it does not certify live payment settlement or webhook delivery.

6. Failure refunds

Implementation `src/utils/supabase/credits.ts`

When processing fails after charging credits, the refund helper checks for an existing refund and inserts a compensating ledger entry. A unique translation-refund index prevents duplicate inserts.

The ledger trigger is the sole balance writer for the refund. An insert error returns failure. The helper's returned balance is a calculation from its read, rather than a guarantee about concurrent balance changes.

SRC/UTILS/SUPABASE/CREDITS.TS · L226-249 LISTING LINES 1-24

```
const newBalance = creditData.amount + params.creditsToRefund;

// The ledger trigger applies the refund; do not update the balance separately.
const { error: transactionError } = await
supabase.from('credits_transactions').insert({
    profile_id: params.userId,
    translation_id: params.translationId,
    credits: params.creditsToRefund,
    type: 'refund',
    description: 'Credit refund for failed translation',
    details: {
        reason: params.reason,
        refunded_at: new Date().toISOString(),
    },
});

if (transactionError) {
    return {
        success: false,
        error: `Failed to record refund: ${transactionError.message}`,
    };
}

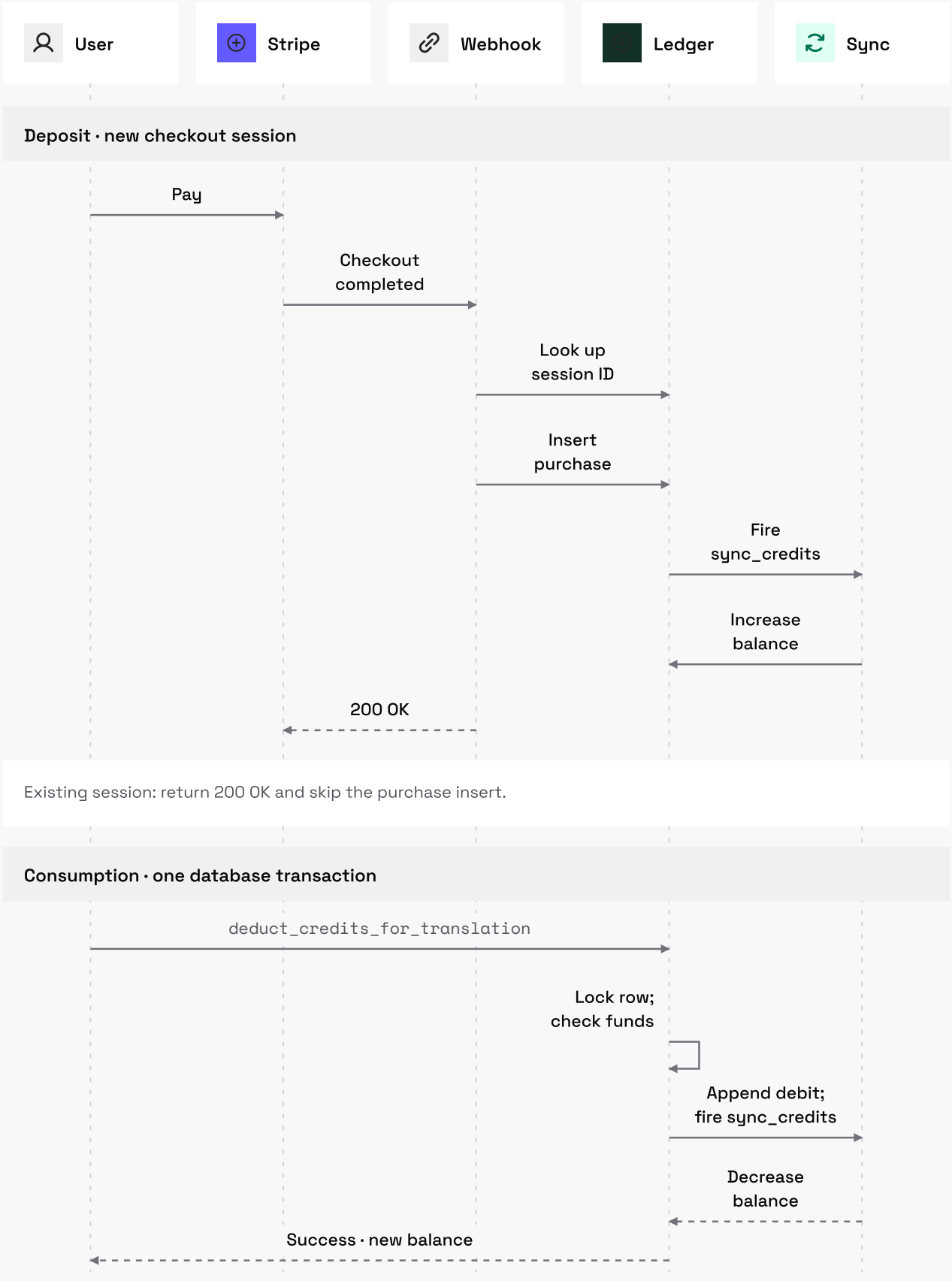
return {
    success: true,
```

REFUND VERIFICATION

The helper checks the ledger insert result, and the unique translation-refund index rejects duplicate refund entries. `tests/credit-refund.test.ts` covers one balance increment, an existing refund and an insert failure. The balance returned by the helper remains an estimate from its read.

7. Financial data flow

05 Financial data flow



8. Pricing rules

Implementation `src/constants/credits.ts`

The ledger stores credit events. The pricing rules determine how many credits a translation consumes and how credit packs are priced.

8.1 Credit calculation

Page rate

`PAGES_PER_CREDIT = 1` and `MINIMUM_CREDITS_PER_TRANSLATION = 1`.

Calculation

`max(1, ceil(pageCount / 1))` for positive page counts. Thus a three-page input requires three credits.

Purchase packs

Credit-pack totals and discounts are configured independently in `src/constants/credits.ts`.

SRC/CONSTANTS/CREDITS.TS · L144–150 LISTING LINES 1–7

```
01 export const calculateCreditsRequired = (pageCount: number): number => {
02     if (pageCount <= 0) {
03         throw new Error('Page count must be greater than 0');
04     }
05
06     return Math.max(MINIMUM_CREDITS_PER_TRANSLATION, Math.ceil(pageCount /
   PAGES_PER_CREDIT));
07 };
```

9. Manual adjustments

Implementation `src/actions/admin/users/adjust-credits.ts`

Support grants and penalties use the ledger's normal write path. Administrative tools should not edit `credits.amount` directly.

9.1 Administrative action

The administrative action inserts a `bonus` or `penalty` transaction. The balance trigger processes that entry through the same ledger path as other recognized events.

SRC/ACTIONS/ADMIN/USERS/ADJUST-CREDITS.TS · L21–43 LISTING LINES 1–23

```
export async function adjustCredits({
    amount,
    description,
    userId,
    type = 'bonus',
}: {
    userId: string;
    amount: number;
    description: string;
    type?: 'bonus' | 'penalty' | 'refund';
}): Promise<ApiResponseOrError<Tables<'credits'>[]>> {
    const logger = getLogger('ACTIONS:ADMIN:USERS:ADJUST_CREDITS');

    try {
        const { user: currentUser } = await validateUser({ requiredRole: 'ADMIN' });

        if (!userId) {
            const errorResponse = standardizeError({
                description: 'User ID is required but missing.',
                message: 'Missing User ID.',
                status: 400,
                success: false,
                title: 'User ID Missing',
```

10. Financial reporting

Implementation `src/actions/admin/analytics/statistics/get-credits-burn-down-analytics.ts`

The transaction timestamps also support usage and purchase reporting.

10.1 Credit usage analysis

Aggregate transaction history to compare purchases and consumption over time.

SRC/ACTIONS/ADMIN/ANALYTICS/STATISTICS/GET-CREDITS-BURN-DOWN-ANALYTICS.TS · L26–45 LISTING LINES 1–20

```
export async function getCreditsBurnDownAnalytics({
    endDate,
    startDate,
}: {
    startDate?: Date;
    endDate?: Date;
} = {}): Promise<ApiResponseOrError<AdminCreditTransaction[]>> {
    const logger = getLogger('ACTIONS:ADMIN:GET_CREDIT_HISTORY');

    try {
        await validateUser({
            requiredRole: 'ADMIN',
        });

        const supabase = await createAdminClient();

        const query = supabase
            .from('credits_transactions')
            .select('credits, type, created_at')
            .order('created_at', { ascending: false });
```

11. Balance refresh and checkout

Implementation

`src/hooks/queries/auth/use-balance-data.ts` & `src/queries/tanstack/billing-query-options.ts`

The interface must refresh its cached balance after relevant events so users can see ledger changes.

11.1 Refresh policy

The documented balance view uses TanStack Query refresh events rather than a dedicated realtime subscription.

Cached balance

Treat the cached balance as stale immediately (`staleTime: 0`).

Refresh events

Refresh the balance when the user returns to the page, after a relevant mutation, and on mount.

Window focus: Refetch when the user returns from another tab, such as Stripe checkout.

Invalidation: Actions such as `adjustCredits` call `revalidatePath`.

Mount: Fetch the current balance when the component mounts.

SRC/QUERIES/TANSTACK/BILLING-QUERY-OPTIONS.TS · L23–31 LISTING LINES 1–9

```
export const balanceDataQueryOptions = (userId?: string | undefined) =>
    queryOptions<number, ApiErrorType, number, ['zustand-balance-store', string | undefined]>({
        gcTime: 30 * 60 * 1000,
        placeholderData: keepPreviousData,
        refetchOnMount: 'always',
        refetchOnReconnect: 'always',
        refetchOnWindowFocus: true,
        queryFn: async () => {
            const response = await getCreditsBalance();
```

11.2 Commerce configuration

The transaction's `details` JSONB stores currency and payment metadata.

MULTI-CURRENCY HANDLING

Currency data

The checkout handler stores currency, conversion and provider payment references in transaction details. Inspect the current handler before calculating a customer's charge.

Provider boundary

Provider fees, exchange rates and document availability are external inputs. The offline suite does not validate billing.

Store the invoice and receipt references returned by Stripe with the transaction. These references help locate the original payment documents; their continued availability depends on the provider.

- `invoice_pdf`: Direct link to the tax invoice.
- `receipt_pdf`: Direct link to the payment receipt.

11.3 First-purchase state

The documented initial account has no free credit grant.

Create the balance

The documented `create_credits_on_profile_insert` trigger creates a `public.credits` row with `amount = 0`.

First purchase

A new account must obtain credits before starting paid translation work.

12. Integrity controls

CONTROL	IMPLEMENTATION	BOUNDARY
Concurrent deduction	Balance row lock and consumption insert	Deduction RPC
Balance projection	Ledger insert invokes `sync_credits`	Database trigger
Ordinary transaction access	Owned records are readable; privileged policies differ	RLS
Webhook authenticity	Stripe event signature verification	Webhook adapter
Duplicate deposits	Session lookup and unique session index	Purchase ledger
Duplicate refunds	Translation lookup and unique translation index	Refund ledger

13. Operational configuration

Provision these environment values and provider integrations for the billing flow.

13.1 Environment dependencies

Implementation `src/env.ts`

ENVIRONMENT VARIABLE	BOUNDARY	PURPOSE
`STRIPE_SECRET_KEY`	Server	Authenticates API calls to Stripe (Charges, Portal).
`STRIPE_WEBHOOK_SECRET`	Server	Verifies signature of incoming webhook events to prevent spoofing.
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Client	Initializes Stripe Elements for the "Top-Up" UI.

13.2 Stripe integration

Stripe-hosted checkout and customer-portal pages are created through application API calls.

Checkout session

Start

The top-up page calls the checkout-session action with the selected credits.

Create

The server creates a hosted Stripe Checkout Session.

Return

The configured success URL is `/account/credits/checkout/{CHECKOUT_SESSION_ID}/success`. Verify the resulting session through the connected provider.

Customer portal

Request

The billing interface calls `openCustomerPortal`.

Create

The server requests a Stripe billing-portal session with a return URL under `/account/billing`.

Configure

Available portal operations depend on the account's Stripe portal settings.

Webhook endpoint and events

Endpoint

`/api/payment/webhook`

Webhook events

`checkout.session.completed`: Credits user balance upon successful payment.

- `checkout.session.async_payment_failed`: Handles failures for delayed payment methods (e.g., SEPA).
- `customer.deleted`: Reserved for future cleanup logic (currently no-op).

PART I · THE ENGINE

Intelligence pipeline 03

Vision processing, orchestration, and document compilation.

IN THIS SECTION

01 Vision-first processing P. 040
02 Durable orchestration P. 042
03 Document translation engine P. 044
04 Responses API adapter P. 048
05 Failure recovery P. 049
06 Visual quality checks P. 051
07 Document compilation P. 052
08 Translation verification P. 053
09 Pipeline observability P. 054
10 Document types P. 057
11 Capacity and runtime P. 060
12 Processing cost estimates P. 061
13 Operational runbook P. 062
14 Architecture decisions P. 064
15 Implementation summary P. 066

Modules	`src/utils/ai/*` · `trigger/*` · `src/config/ai.ts`

1. Vision-first processing

The translation path uses page images to interpret content and structure before rebuilding an editable document.

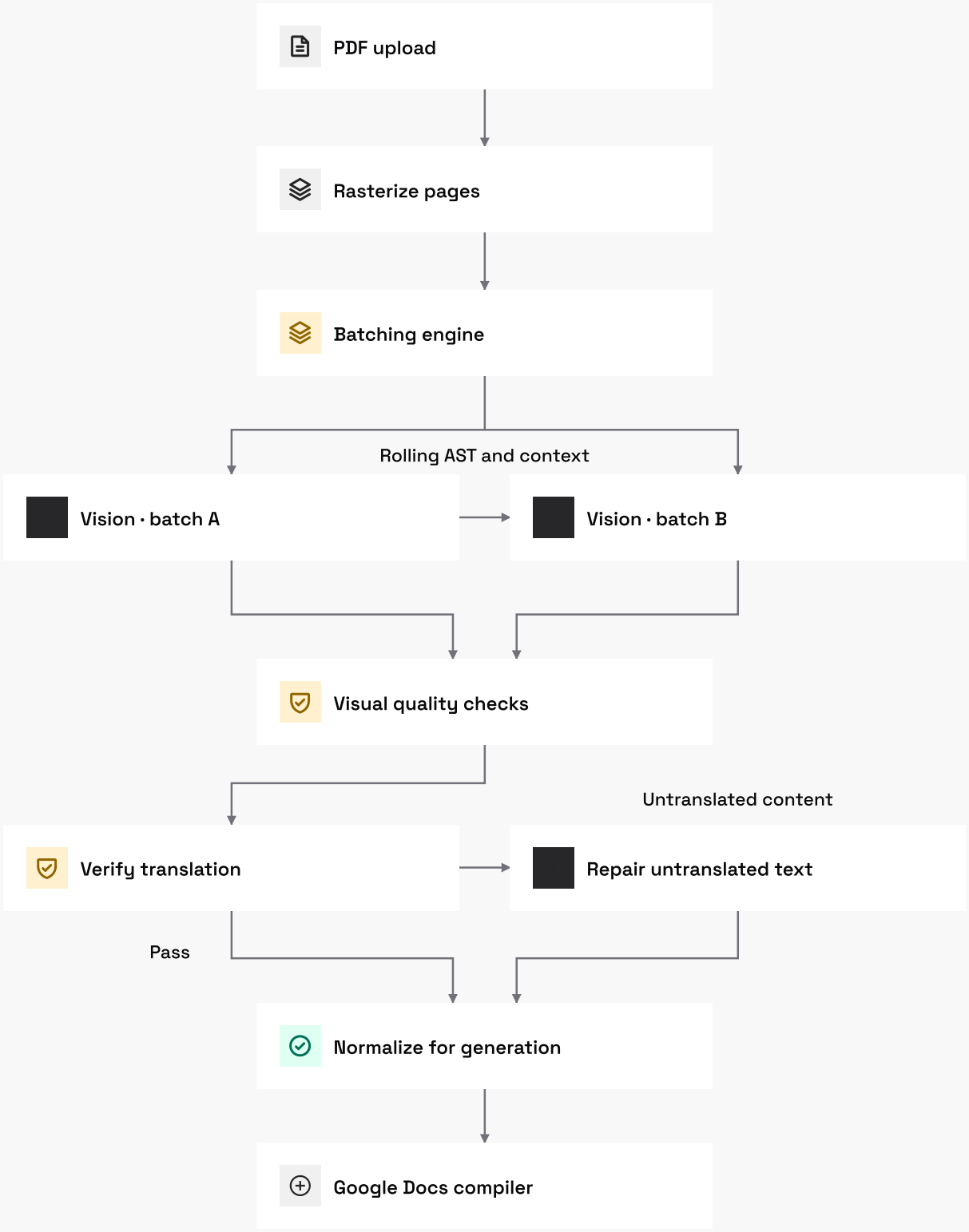
The pipeline uses page images to recover both text and layout. Its output is an editable document structure that can be reviewed and corrected before delivery.

CONFIGURED MODEL ROLES

`src/config/ai.ts` assigns GPT-5.2 to translation, QA and verification, and GPT-5-mini to metadata and repair. These are adapter settings, not measured latency or cost claims. Batch context is returned with vision output and merged locally.

1.1 Pipeline architecture

06 Vision architecture



2. Durable orchestration

Implementation `trigger/tasks/partials/vision-translate-document.ts`. The parent workflow is `trigger/tasks/translation-task.ts`.

`visionTranslateDocumentTask` coordinates four weighted processing stages on Trigger.dev:

Prepare

0–15%: download, normalize and collect image metadata.

Translate

15–65%: translate page batches using the configured translation model.

Review

65–90%: run visual QA with the configured QA model.

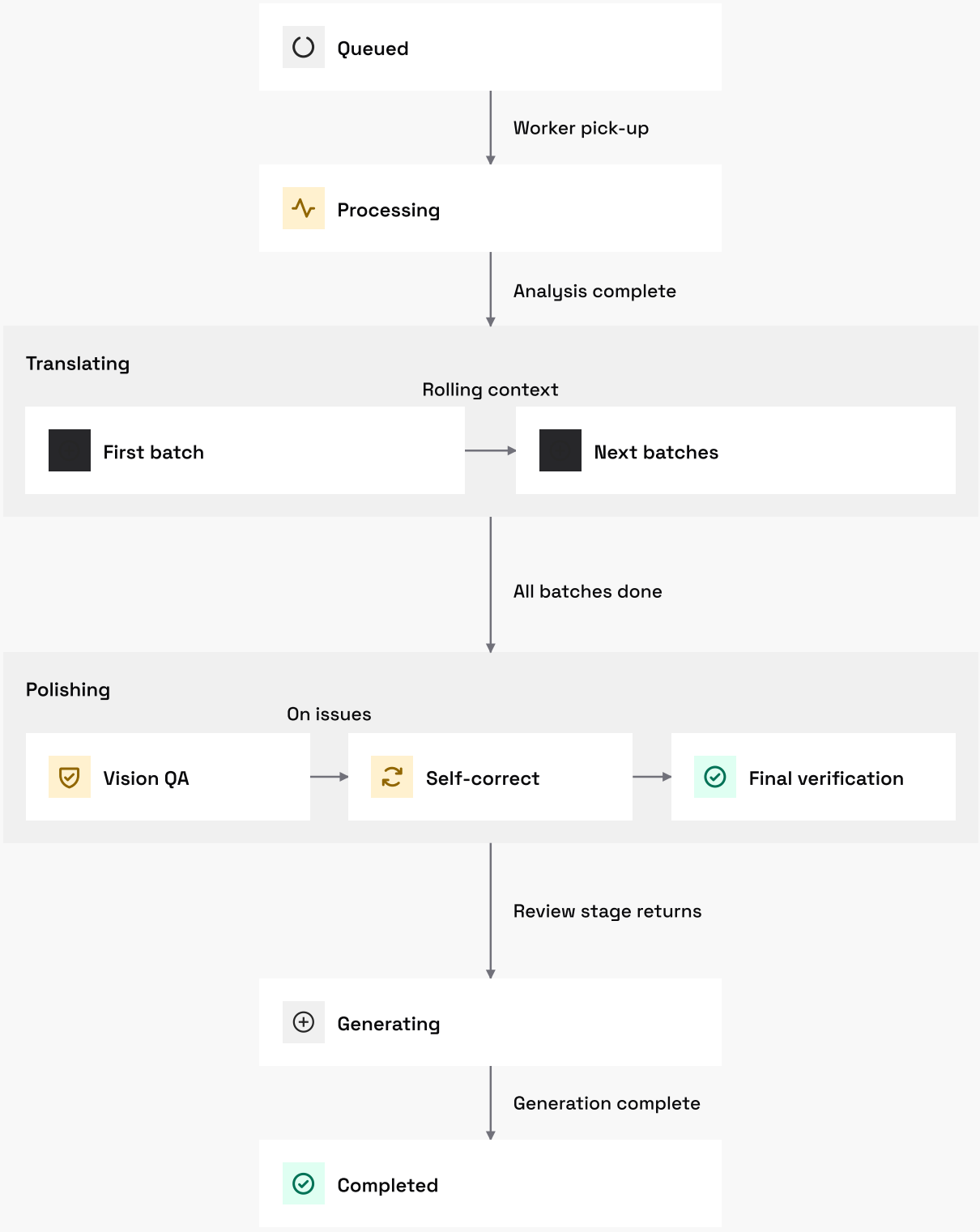
Generate

90–100%: compile and export. These are progress ranges, not measured time proportions.

2.1 Lifecycle states

The lifecycle figure groups processing stages. Inspect the persisted status enum, task branches and failure handlers for the complete state behavior.

07 Pipeline lifecycle



Lifecycle states drive progress updates while Trigger.dev runs the background workflow. The job can outlive a frontend HTTP request; its deadline, retry policy, and checkpoints are configured separately.

2.2 Execution pipeline

`generateDocsTask` updates generation status, waits for document generation and then invokes finalization. The parent translation task coordinates translation and the generation path.

TRIGGER/TASKS/GENERATE-DOCS-TASK.TS · L66–88 LISTING LINES 1–23

```
const generateResult = await logger.trace('generate-google-doc', async () => {
    return await generateDocumentTask.triggerAndWait({
        translationId: payload.translationId,
        userId: payload.userId,
    });
});

// Log the export status from generation
if (generateResult.ok) {
    const exportData = generateResult.output?.data;
    logger.info('[DOCS] Document generation completed', {
        translationId: payload.translationId,
        hasDocx: !!exportData?.uploadedDocumentUrls?.downloadUrl,
        hasPdf: !!exportData?.uploadedDocumentUrls?.pdfDownloadUrl,
        hasHtml: !!exportData?.uploadedDocumentUrls?.htmlDownloadUrl,
        hasGoogleDocs: !!exportData?.uploadedDocumentUrls?.googleDocsEditUrl,
        fromCheckpoint: exportData?.fromCheckpoint ?? false,
    });
} else {
    logger.error('[DOCS] Document generation subtask failed', {
        translationId: payload.translationId,
        error: generateResult.error,
    });
```

3. Document translation engine

Implementation `src/utils/ai/vision-document-translator.ts`

The translation module calls the model through `src/utils/ai/providers/openai-adapter.ts`.

3.1 Translation instructions

The system instructions define how to treat names, entities, legal terminology, and document structure. These are translation rules for review, not a certification of the output.

Identify content

”Does this text identify WHO or WHERE?” (If yes -> PROTECT/Do Not Translate).

Describe content

”Does this text describe WHAT something is?” (If yes -> TRANSLATE).

Legal entities

Explicit rules for Government Bodies, Academic Titles, and Legal Terminology.

3.2 Input checks and normalization

Before translation, normalize the input and collect the metadata used for batching.

Normalize input

Convert supported source inputs into page images for the vision path.

Describe the file

A metadata pass proposes a descriptive filename from the source.

Size batches

Estimate density and apply bounded batch-size heuristics.

3.3 Contextual batching

Each batch receives context from earlier pages to reduce inconsistent terminology and broken document hierarchy:

Entity context

Pass names already encountered, such as people and organizations, to reduce inconsistent treatment between batches.

Terminology context

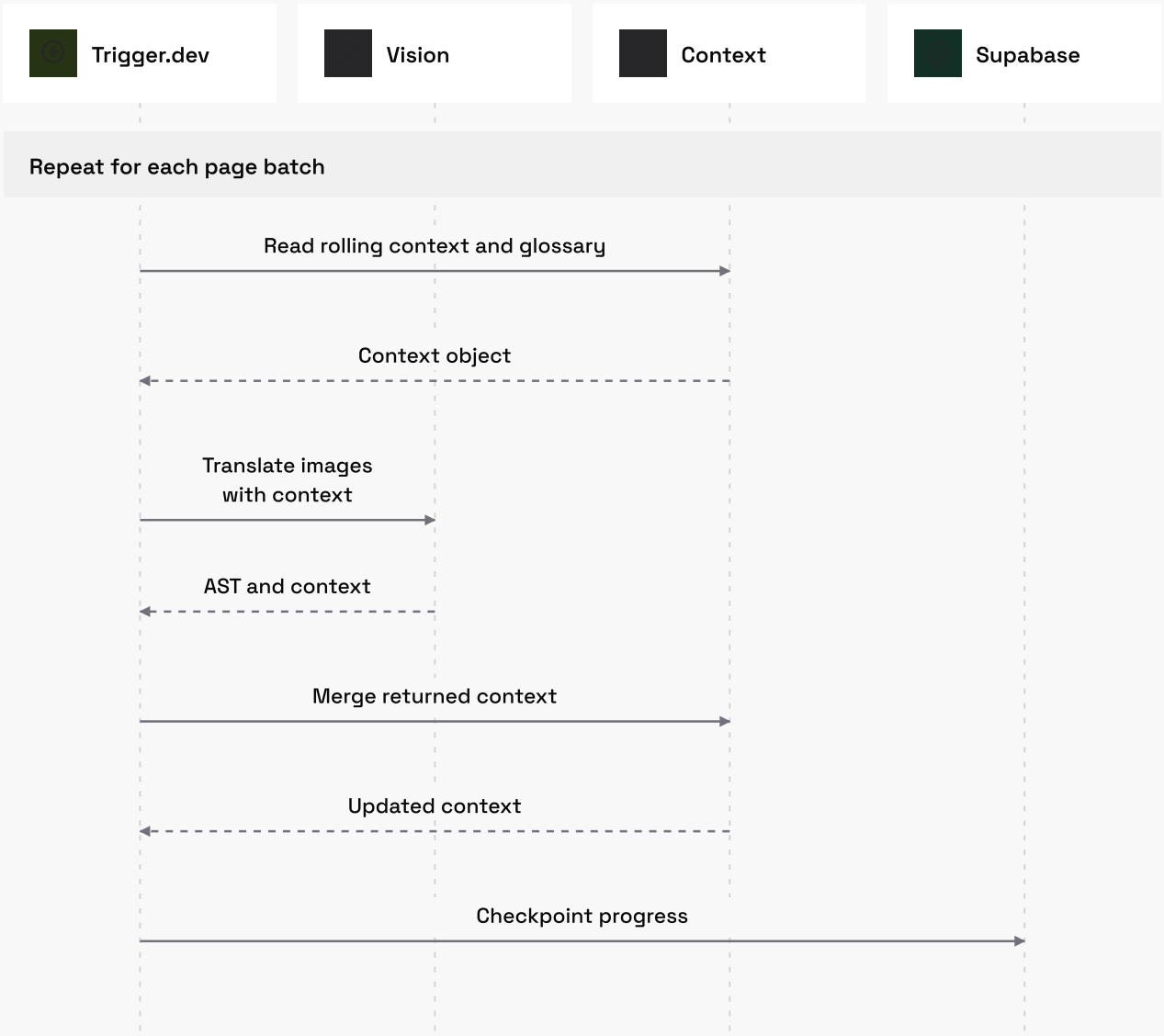
Pass the selected glossary and legal terms to support consistent wording.

Structural context

Pass preceding headings and document structure so later batches can continue the hierarchy.

This rolling context lasts for the task. It is not evidence of a separate retrieval index or persistent knowledge base.

08 Contextual batching



Vision uses the configured translation model.
Context returned in the same response is merged locally for the next batch.

SRC/UTILS/AI/VISION-DOCUMENT-TRANSLATOR.TS · L542–551 LISTING LINES 1–10

```
export type BatchContext = {
    /** Entities found so far (names, companies, addresses) */
    entities: string[];
    /** Key terminology translations for consistency */
    terminology: Record<string, string>;
    /** Last few headings for structure continuity */
    recentHeadings: string[];
    /** Summary of document structure so far */
    structureSummary: string;
};
```

4. Responses API adapter

Implementation `src/utils/ai/providers/openai-adapter.ts`

The adapter uses `openai.responses.create`. The relevant controls are reasoning effort, output detail, and HTTP transport.

Model settings

The adapters use reasoning and text verbosity settings from `src/utils/ai/ai-config.ts`. Accepted settings depend on the selected provider model.

Output budget

Verbosity is distinct from the output-token limit. Malformed and truncated output still requires validation.

Transport

An Undici agent has explicit header, body and keep-alive budgets. These do not override provider or proxy limits.

SRC/CONFIG/AI.TS · L5–14 LISTING LINES 1–10

```
export const AI_CONFIG = {
    /**
     * The primary model for high-intelligence tasks (Translation, QA, Verification).
     * @see https://openai.com/index/introducing-gpt-5-2/
     */
    model: 'gpt-5.2',

    /**
     * The fast/cost-effective model for simple tasks (Metadata, Repair).
     */
```

5. Failure recovery

5.1 JSON recovery

The JSON recovery utility attempts to repair malformed output. Validate the recovered structure before accepting it; repair cannot recreate missing source content reliably.

Repair syntax

Try supported JSON recovery paths, including unclosed containers and recoverable content nodes.

Validate content

Recovery cannot reconstruct omitted source text. The subsequent schema and review checks determine whether a result is usable.

5.2 Checkpoints and retries

Before starting translation, the workflow checks `translation.details.translated_document` for a reusable checkpoint.

Resume checkpoint

If a usable `translation.details.translated_document` checkpoint exists, reuse it rather than repeating translation after a later export failure.

Cost boundary

Checkpoint reuse can avoid repeated inference. Verify checkpoint persistence, concurrent-run handling, and credit settlement before treating retries as cost-free.

5.3 Dynamic batch sizing

Implementation `batch-sizing.ts`

The pipeline varies batch size according to estimated document density.

Default cap

The batch-sizing helper caps simple, medium and complex cases at five, three and two pages respectively.

Token headroom

The default budget uses `floor(16384 × 0.7) = 11468` estimated output tokens. These are code heuristics, not measured reliability or billing values.

5.4 Refund on processing failure

Implementation `trigger/tasks/translation-task.ts`

A processing failure initiates a compensating credit transaction. Refund completion must be confirmed by the database write; catching an exception alone does not prove that the balance was restored.

TRIGGER/TASKS/TRANSLATION-TASK.TS · L249–258 LISTING LINES 1–10

```
const refundResult = await refundCreditsForFailedTranslation(supabase, {
    userId: payload.userId,
    translationId: payload.translationId,
    creditsToRefund: transaction.credits,
    reason: errorMsg || 'Translation task failed',
});

if (refundResult.success) {
    span.setAttribute('creditsRefunded', transaction.credits);
    span.setAttribute('newBalance', refundResult.newBalance || 0);
```

Failure handling also sends a notification that links back to the affected translation.

Authenticated destination

Failure notifications link to `/app/translations/:id`. The route must enforce the identity and tenant controls described in System 1.

5.5 Schema validation

Implementation `schema-validation.ts`

Validate each JSON response against the Zod schema before passing it to later stages.

Validate schema

`parseAndValidateJSON(schema, output)`

Reject bad output

If the AI returns a valid JSON that misses a required field (e.g. `content_type`), the system throws a specific `SchemaValidationError` rather than polluting the database with corrupt data.

6. Visual quality checks

Implementation `src/utils/ai/vision-document-qa.ts`

A second model pass compares the source with the generated document and proposes corrections. Human review remains necessary for translation accuracy and legal use.

6.1 Compare source and output

The visual QA pass receives two inputs:

- The **Original Image** (The Ground Truth).
- The **Translated JSON** (The Candidate).

The QA pass compares source images with the generated document and proposes changes to layout and content. Treat its output as a review aid.

Review checks may compare heading alignment or merged-cell spans with the source image. Their output is model-dependent; inspect the structured issues and the actual document rather than treating these examples as recorded model responses.

6.2 Progress reporting

The QA layer calls `report_progress` to send structured status messages to the frontend. These messages describe the current task; they are not a transcript of the model's internal reasoning.

The worker updates `stage`, `current_operation` and `progress` metadata. Read the run's actual values when diagnosing processing; this publication does not reproduce a production run.

These activity messages give the user more useful feedback than an undifferentiated loading state.

6.3 Fallback when polishing fails

Implementation `trigger/tasks/partials/polish-translation.ts`

An optional QA or polishing failure falls back to the available translated checkpoint. This fallback concerns output quality; it does not relax authentication or tenant controls.

TRIGGER/TASKS/PARTIALS/POLISH-TRANSLATION.TS · L182–191 LISTING LINES 1–10

```
} catch (error) {
    const errorMsg = (error as Error).message || 'Unknown error';

    metadata.set('stage', 'error');
    metadata.set('current_operation', 'Failed');
    metadata.set('error', errorMsg.substring(0, 200));

    logError('Polish pass failed', error, {
        id: payload.translationId.substring(0, 8),
    });
```

If the optional polishing stage fails, the workflow can return the translated checkpoint with a warning. Earlier failures may still prevent document delivery.

7. Document compilation

Implementation `src/utils/gcp/google-docs/mapper.ts`

The `generate-document` stage compiles the shared document tree into the export representation.

7.1 AST-to-Docs mapping

The chosen compiler uses Google Docs for document layout and format export.

Document input

Accept the `NormalisedDocument` AST.

Compilation

`GoogleDocsMapper` translates nodes into `batchUpdate` requests.

Tables: `insertTable` creates the grid; `updateTableCellStyle` applies cell rules and borders.

Headings: `insertText` adds content; `updateParagraphStyle` applies a named heading style.

Images: `insertInlineImage` inserts an image at a document index. Positioned-image handling is a separate concern.

7.2 Export formats

After compilation, the Drive export flow produces the required file formats. Completion and availability depend on the export request and storage write succeeding.

DOCX

Editable delivery for Microsoft Word.

PDF

Fixed-layout output for review and distribution. Export alone does not certify a translation.

HTML

Lightweight browser viewing.

7.3 Stored outputs and delivery

Download generated artifacts and write them to application storage for authenticated delivery. The user-facing download does not depend on access to the live Google Doc.

Storage

Generated objects use the `exports` bucket and paths of the form `exports/{translationId}/{fileId}{extension}`.

Delivery

The generator requests signed URLs. Their expiration and the recipient's possession of a URL govern access; it does not require a Google account.

Availability

DOCX generation is required by this task. Secondary formats report their individual results.

8. Translation verification

Implementation `src/utils/ai/verification/verify-translation.ts`

Verification checks inspect the generated structure before delivery and flag content that may need repair.

8.1 Untranslated content detection

A language heuristic looks for blocks that appear to remain in the source language.

8.2 Repair pass

The detector and repair adapter inspect potentially untranslated blocks. Thresholds and exemptions are implementation heuristics; they do not establish a language-coverage or accuracy percentage.

8.3 Deliver with review warnings

Recoverable verification warnings do not abort the documented workflow. Preserve the result and make unresolved issues visible for review.

Editable result

Recoverable QA failures may retain the translated checkpoint for human review.

Warning boundary

Inspect audit data and the visible review state. Do not treat successful export as proof that every issue is resolved.

9. Pipeline observability

OPERATOR EVIDENCE

Use run phases, provider errors, checkpoints and database state together when investigating work. Logs do not replace review of the resulting document.

Trigger.dev traces expose processing phases and events in an operator interface.

9.1 Progress telemetry

The documented progress design publishes task metadata for a live frontend view. System 4 separately describes the Supabase translation subscription.

Run tags

Task helpers attach identifiers and language or page context for filtering. Treat run data as operational information.

Progress

Task metadata and Supabase translation updates report phases and completion estimates.

Timing

Spans measure the current run. This handbook makes no latency benchmark claim.

Logs

Structured phase events aid investigation. Do not log credentials or document contents.

9.2 Logging implementation

Implementation `logging.ts`

Use stable phase names so logs from different processing stages can be compared.

TRIGGER/LOGGING.TS · L61-70 LISTING LINES 1-10

```
export function logPhase(phase: LogPhase, message: string, data?: Record<string, unknown>): void {
    const metrics = data ? ` | ${formatMetrics(data)}` : '';
    logger.info(`[${phase}] ${message}${metrics}`);
}

/**
 * Logs a summary at the end of a phase.
 * Use for aggregated metrics after completing a batch of operations.
 *
 * @param operation - What completed (e.g., "OCR processing", "Translation")
```

9.3 Audit data

Implementation `src/types/ai-audit.ts`

A `QAStageAudit` object records the QA fields described below in a persistent JSONB column. This supports investigation; the excerpt does not establish complete decision capture or a legal compliance guarantee.

SRC/TYPES/AI-AUDIT.TS · L219–231 LISTING LINES 1–13

```
export type AIAuditData = {
    schema_version: 1;
    /** Processing stage audit (document download, conversion, metadata extraction) */
    processing?: ProcessingStageAudit;
    /** Translation stage audit (always present for completed translations) */
    translation: TranslationStageAudit;
    /** QA stage audit (present if QA was run) */
    qa?: QAStageAudit;
    /** Metadata stage audit (present if metadata was generated) */
    metadata?: MetadataStageAudit;
    /** Pipeline-level summary */
    pipeline: PipelineSummary;
};
```

9.4 Input provenance

Processing metadata records the input path, processing actor, and upload time:

Input path

Processing metadata can identify the source object within application storage.

Actor and time

Actor and timestamp fields support investigation when populated. They are operational metadata, not evidence of a complete chain of custody.

9.5 Progress history

`progress_history` stores the documented rolling window of 50 state changes:

Timestamp

When the progress event was recorded.

Completion

The reported percentage at that point in the job.

Activity

A user-facing label such as “Analyzing table structure”.

State transition

The change in job state. This history helps reconstruct the processing sequence.

10. Document types

10.1 Normalized document tree

`NormalisedDocument` is the recursive document tree shared by translation, editing, and compilation. Nodes represent content and layout independently of the export format.

Source `src/engine/document/types.ts`

SRC/ENGINE/DOCUMENT/TYPES.TS · L20–43 LISTING LINES 1–24

```ts
export type NormalisedDocument = {
    /**
     * The type of the root node, which should always be 'doc'.
     */
    type: 'doc';

    /**
     * An array of content nodes representing the structure and content of the document.
     */
    content: ContentNode[];

    /**
     * An array of translated content nodes in the target language.
     */
    translatedContent?: ContentNode[];

    /**
     * The type of document (e.g., 'invoice', 'certificate', 'resume', 'passport').
     */
    documentType: string;

    /**
     * Languages detected in the document, along with their confidence scores.
     */
```

Refinement schema

Structured Outputs constrains the response shape, and Zod validates it against the refinement schema. Structural validity does not establish translation accuracy. Implementation: `src/schemas/document/refinement-schema.ts`.

10.2 Branding and PDF types

Typed branding settings describe watermarks, stamps, and page layout for generation.

Source `src/types/document/branding-settings.ts`

SRC/TYPES/DOCUMENT/BRANDING-SETTINGS.TS · L86-100 LISTING LINES 1-15

```
export type DocumentBrandingSettings = {
    enabled: boolean;
    logoUrl: string | null;
    companyName: string | null;
    headerText: string | null;

    footerText: string | null;
    showCertificationStamp: boolean;
    certificationNumber: string | null;
    certificationStampUrl: string | null;
    certificationStampPlacement: CertificationStampPlacement;
    certificationStampSize: CertificationStampSize;
    showPageNumbers: boolean;
    watermarkText: string | null;
    watermarkPreset: 'light' | 'dark' | 'off';
```

11. Capacity and runtime

11.1 Page-count limit

The translation parent and vision task set `maxDuration: 86400`, which is `86400 / 3600 = 24` hours. A time budget does not establish a supported page count or completion guarantee.

DOCUMENT CONCERN	WORK PERFORMED	EVIDENCE BOUNDARY
Small text	Image resolution and visual review	Evaluate legibility on representative inputs.
Merged tables	Grid repair and native table mapping	Inspect row and column spans after export.
Long documents	Batching, checkpoints and worker budget	No measured maximum document size is claimed.
Mixed scripts	Direction, language instructions and human review	No universal language-coverage claim is made.

CAPACITY ASSUMPTION

Page count, image detail, batching, retries, and provider response time all affect duration. Measure representative documents before setting an operational commitment.

11.2 Long-running execution

ADR 0002 defines the durable task boundary. Trigger.dev jobs execute document processing separately from the frontend request; task and provider limits still apply.

Task budget

Translation tasks request 86400 seconds; `trigger.config.ts` sets a separate 10000-second default for other tasks.

HTTP budget

The Responses client sets header and body budgets to 7200000 milliseconds. Keep-alive controls idle connection reuse.

Retry boundary

A stored translated checkpoint may avoid repeating translation after a later failure. Concurrent runs and settlement require separate checks.

12. Processing cost estimates

Image-based translation consumes vision input, text context, and output tokens. The estimate below separates these cost drivers.

12.1 Vision input cost

The source budget separates image input from text context and output. Higher image detail can help with small text in stamps and footnotes, at additional token cost.

Image-token calculation

Use provider-reported token usage and the rates for your chosen model. Image accounting is model-specific; this handbook does not assume a fixed tile price.

Input usage

Let I be billed image and text input tokens for the run.

Output usage

Let O be billed output tokens; include additional billed categories under the provider's terms.

Rates

Let r_i and r_o be the applicable prices per million tokens.

Per-page estimate

COMPONENT	CALCULATION	SOURCE OF INPUT
Input cost	$I \times r_i / 1000000$	Provider usage and applicable rate
Output cost	$O \times r_o / 1000000$	Provider usage and applicable rate
Run total	Input + output + other billed categories	Include retries, QA and repair calls
Per-page allocation	Run total / processed pages	An allocation, not a quoted page price

COST BOUNDARY

Compute, storage, export services and human review add costs beyond model tokens. Verto credits are a product rule and are not a measurement of provider cost or time saved.

12.2 Complexity assumptions

The batch-sizing helper applies document-type multipliers to estimated output size. These affect batching, not the customer credit formula or a measured inference-cost multiplier.

General

Output-size multiplier `1.0`.

Legal

Output-size multiplier `2.2`.

Financial

Output-size multiplier `3.0`. Values come from `src/engine/translation/batch-sizing.ts`.

13. Operational runbook

Use this sequence to investigate translation runs and choose a recovery action.

13.1 Investigate a stalled job

Investigate a run whose progress and recent events stop changing. A progress percentage is a stage estimate, not elapsed-time evidence.

Locate the run

Inspect the task run for the affected translation in your configured Trigger.dev project.

Inspect evidence

Check concurrency, phase logs, checkpoints and terminal state before retrying.

Retry deliberately

Read `src/actions/services/translations/retry-translation.ts`, `retry-from-stage.ts` and `retry-format-export.ts`. Authorize the caller and avoid overlapping active runs.

13.2 Error responses

ERROR PATTERN	INTERPRETATION	OPERATOR ACTION
Rate limit	Provider capacity or quota	Inspect classified retry policy and provider limits.
Context length	Request exceeds an accepted budget	Review batch sizing and output requirements.
Invalid input	Unreadable or unsupported source	Inspect the source without exposing private content.

14. Architecture decisions

The numbered ADRs in `docs/decisions/` state the requirements, chosen boundaries, consequences and alternatives. These summaries use the same numbering.

14.1 Generative vision

ADR 0001 Accepted

REQUIREMENT

Translate text and represent headings, tables and reading order in a shared editable document tree.

Alternative: AWS Textract

Boundary

OCR text and geometry would require an additional reconstruction layer. The ADR does not claim a measured coverage or price comparison.

Alternative: Google Document AI

Boundary

OCR structure would still need mapping into the shared document representation. No benchmark comparison is claimed.

Selected: generative vision

Capability

Page images and instructions provide text and layout context to the model.

Tradeoff

Output is probabilistic and needs validation and human review. Latency and cost depend on inputs and provider behavior.

DECISION

Use vision translation with the shared document tree. Keep schema checks, table repair and reviewer control explicit.

14.2 Durable execution

ADR 0002 Accepted

WORKLOAD BUDGET

For `P` pages and an assumed serial time `t` seconds per page, `T = P × t`. This formula is a planning model, not a measured latency result. Batching, retries, QA and export alter end-to-end duration.

ALTERNATIVE OR CHOICE	EXECUTION BOUNDARY	CONSEQUENCE
Request-bound functions	Host invocation limits	Long work needs checkpoints and orchestration.
Inngest	Workflow orchestration alternative	Evaluate host runtime and state boundaries for the workload.
Trigger.dev	Selected durable task boundary	Worker execution and tracing remain separate from frontend requests.

DECISION

Run document processing in Trigger.dev tasks. Keep user requests short and persist state needed for recovery.

14.3 Document compilation

ADR 0003 Accepted

REQUIREMENT

Produce editable DOCX files and fixed-layout PDF records from the same document representation.

ALTERNATIVE OR CHOICE	EVALUATION BOUNDARY
Browser rendering	Requires its own pagination, font and document-layout implementation.
Google Docs API	Maps the shared tree into native document operations and Drive exports.

DECISION

Use Google Docs and Drive for native document generation and configured exports. Accept provider access, quotas and layout differences as explicit constraints.

15. Implementation summary

CAPABILITY	IMPLEMENTATION	REVIEW BOUNDARY
Ingestion	Page images and vision	Check source legibility.
Translation	Document context and structured output	Human language review remains necessary.
Quality checks	Schema, grid repair and QA adapters	Validity is distinct from accuracy.
Progress	Task metadata and database events	Percentages describe stages.
Audit	Typed processing and QA data	No legal compliance guarantee is made.

PART II · THE EXPERIENCE

User workspace

04

The editor, source viewer, client records, and realtime state.

IN THIS SECTION

01 Workspace responsibilities P. 068

02 Translation editor P. 069

03 Source document viewer P. 077

04 Client records P. 079

05 Realtime synchronization P. 080

06 Workspace resilience P. 083

07 Workspace technology P. 084

08 Performance strategies P. 084

09 Editor decisions P. 085

10 User credit consumption P. 086

Modules	`src/components/translations/translation-editor/*` · `src/app/(protected)/app/customers`

1. Workspace responsibilities

The browser workspace combines translated content, the original source, client records, and editing controls.

It supports two review responsibilities:

Document layout

How to edit complex, layout-preserved documents without breaking the visual hierarchy?

Agency workflow

How to allow our users (translators) to manage *their* own clients within our platform?

1.1 Core editor component

Implementation `TranslationEditor`

The React editor receives `initialContent` as JSON and emits `onChange` events.

SRC/COMPONENTS/TRANSLATIONS/TRANSLATION-EDITOR/TRANSLATION-EDITOR.TSX · L35-45 LISTING LINES 1–11

```
type TranslationEditorProps = {
    initialContent: JSONContent | null;
    localOnly?: boolean;
    onChange: (content: JSONContent) => void;
    editable?: boolean;
    originalContent?: JSONContent | null;
    onReset?: () => void;
    isSaving?: boolean;
    lastSaveTime?: Date | null;
    hasUnsavedChanges?: boolean;
    /**
```

2. Translation editor

Implementation `src/components/translations/translation-editor/translation-editor.tsx`

Tiptap provides a headless editor built on ProseMirror's schema. Verto extends that schema to edit the document structures produced by the translation pipeline.

EDITING CONTRACT

Document model

Tiptap edits a JSON document tree governed by ProseMirror's schema. Content attributes must survive parsing and serialization.

Custom structures

Extensions represent description lists, footnotes, signature blocks, and tables in the editor's document model.

Toolbar state

Selectors subscribe toolbar controls to the editor state they use. Rendering behavior must be profiled in the full component tree.

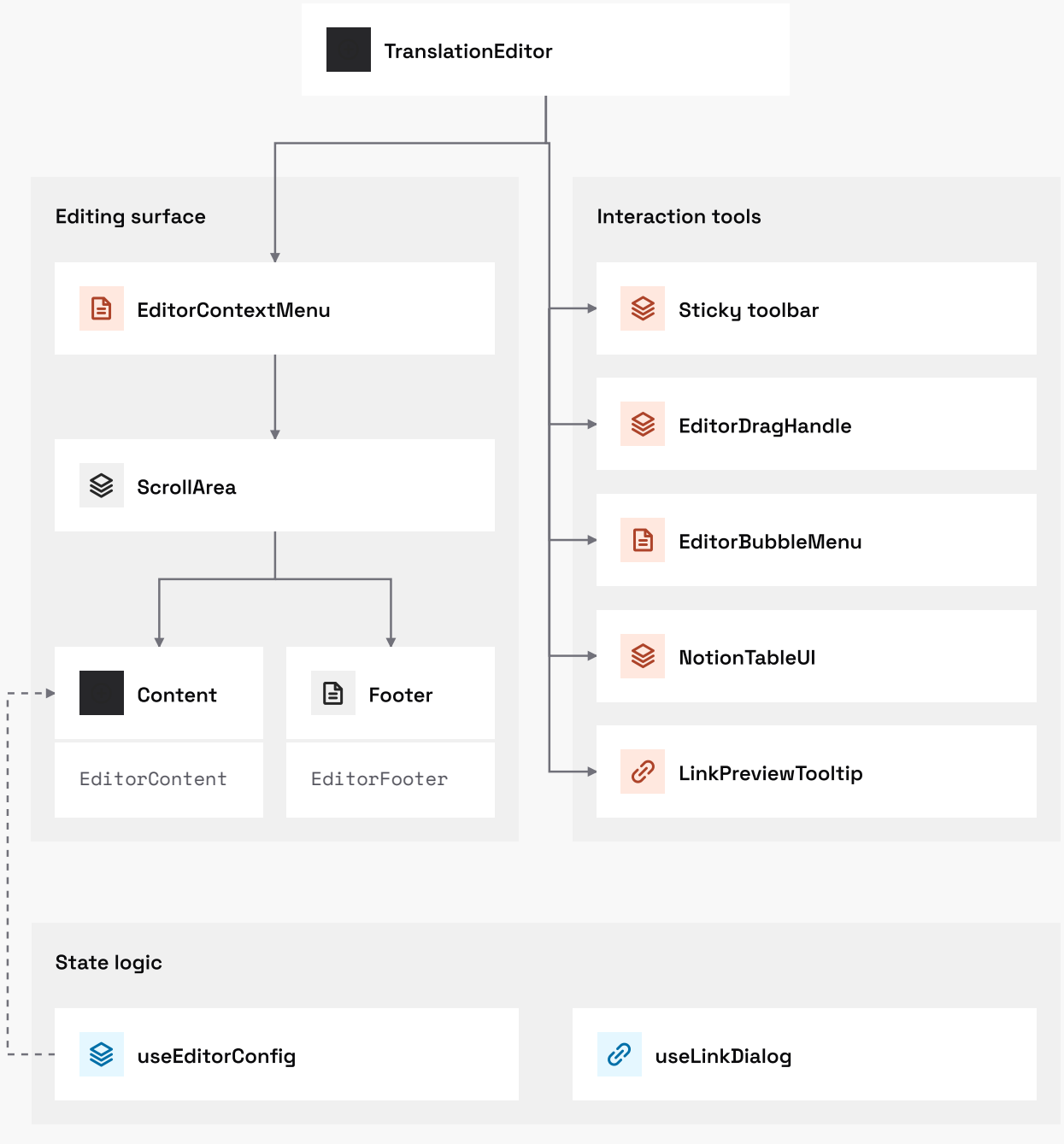
Regeneration

The documented regeneration flow applies saved user edits, then recompiles the document without rerunning translation.

2.1 Component composition

`TranslationEditor` composes specialized components for document editing and review.

09 Editor composition



Solid connectors show component ownership. Dashed connectors show configuration.

Configuration

`useEditorConfig` builds extensions and sanitizes the document at the editor boundary.

Tables

Custom table controls provide row/column operations and merged-cell editing.

Loading

`EditorSkeleton` reserves an editor-shaped surface while initialization runs.

2.2 Document schema extensions

Verto needs domain-specific structures such as signature blocks and description lists. Custom schema extensions represent those structures and connect them to the editor's rendering and serialization.

Structure

`DescriptionList` (DL/DT/DD), `DescriptionItem`, `Footnote`

Formatting

`HeadingExtended`, `ParagraphExtended`, `Spacer`, `ThematicBreak`

Tables

`TableExtended`, `TableHeaderExtended`, `TableCellExtended`

Decorators

`LinkPreviewTooltip` (Hover cards for hyperlinks)

Implementation `src/components/translations/translation-editor/extensions/custom/*`

EXTENSION	REPRESENTATION	BOUNDARY
TableExtended	Table cells, spans and optional borderless presentation	Review merged-cell mapping.
DescriptionList	Semantic description terms and values	Represent paired content without relying on spaces.
ThematicBreak	Visible break node	A review hint, not a guarantee of identical export pagination.
Footnote	Reference nodes	Review references after generation.
Spacer	Vertical spacing node	Output spacing depends on the renderer.
HeadingExtended	Heading levels and attributes	Shared tree structure drives editor rendering.

2.3 Editor state

Implementation `src/components/translations/translation-editor/hooks/use-toolbar-state.ts`

`useEditorState` selects the editor state needed by the toolbar:

SRC/COMPONENTS/TRANSLATIONS/TRANSLATION-EDITOR/HOOKS/USE-TOOLBAR-STATE.TS · L87–110 LISTING LINES 1–24

```
export function useToolbarState(editor: Editor | null): ToolbarState | null {
    return useEditorState({
        editor,
        selector: ({ editor: e }): ToolbarState | null => {
            if (!e) return null;

            return {
                // Text formatting
                isBold: e.isActive('bold'),
                isItalic: e.isActive('italic'),
                isUnderline: e.isActive('underline'),
                isStrike: e.isActive('strike'),

                // Text alignment
                isAlignLeft: e.isActive({ textAlign: 'left' }),
                isAlignCenter: e.isActive({ textAlign: 'center' }),
                isAlignRight: e.isActive({ textAlign: 'right' }),
                isAlignJustify: e.isActive({ textAlign: 'justify' }),

                // RTL State
                isRTL: getIsRTL(e),

                // Lists
                isBulletList: e.isActive('bulletList'),
```

The selector narrows the toolbar's subscription to the state it uses. Re-render behavior also depends on selector equality and the surrounding component tree; profile the full editor before claiming a performance result.

2.4 Regeneration flow

Implementation `src/actions/services/translations/regenerate-document.ts`

The documented editing flow keeps user changes distinct from the generated document and passes them into regeneration.

Edit

Store the edited document tree in `translations.details.user_edits`.

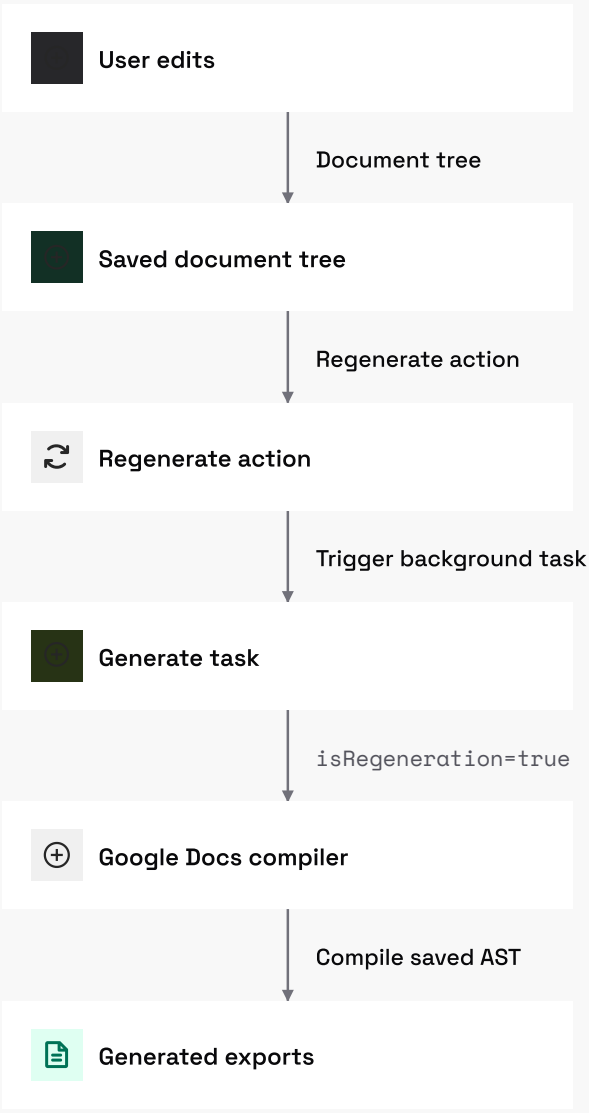
Regenerate

The authorized regeneration action triggers generation with `isRegeneration: true`.

Compile

The generator prefers saved user edits and compiles through Google Docs; this avoids a new vision-translation pass but still uses generation services.

10 Regeneration loop



Regeneration compiles the saved document tree.
The regeneration flag selects this path instead of a new vision translation.

2.5 Brand configuration

Implementation `src/components/branding/branding-editor.tsx`

The branding editor configures headers, footers, watermarks, and stamps separately from the document body.

Component architecture

Its reduced Tiptap schema permits bold, italic, and links through a custom `BubbleMenu`, keeping document-level blocks outside this field.

SRC/COMPONENTS/BRANDING/BRANDING-EDITOR.TSX · L62–73 LISTING LINES 1–12

```
export function BrandingEditor({
    content,
    onChange,
    placeholder = 'Enter text...',
    className,
    disabled = false,
}: BrandingEditorProps) {
    // Create minimal extensions
    const extensions = useMemo(
        () => [
            StarterKit.configure({
                // Disable everything except basic text formatting
```

Pipeline integration

The branding UI produces the `DocumentBrandingSettings` consumed by the pipeline.

Edit settings

The branding editor emits structured header/footer content.

Account configuration

Profile fields hold document branding defaults such as header, footer, logo, watermark and stamp settings.

Compile

The generation path builds `DocumentBrandingSettings` for the Google Docs mapper.

2.6 Editor interactions

Implementation

`src/components/translations/translation-editor/extensions/table-keyboard-shortcuts.ts`

Custom keyboard handlers support common table and content-editing actions:

INTERACTION	SHORTCUT	IMPLEMENTATION
Table Navigation	`Tab` / `Shift+Tab`	Overrides default focus behavior to move cursor between cells (Excel-style).
Markdown Paste	`Ctrl+V`	Detects Markdown syntax on paste (e.g. `**bold**`, `## Header`) and auto-formats it via `PasteMarkdown` extension.
Smart Links	`Cursor`	`LinkPreviewTooltip` appears when the caret is placed inside a link, allowing "Edit/Remove" without breaking flow.

3. Source document viewer

Implementation `src/components/translations/pdf-viewer.tsx`

The original PDF remains the review reference. The source viewer virtualizes pages to limit mounted canvases and browser memory during long-document review.

3.1 Page virtualization

`react-window` and `VariableSizeList` mount the visible page range rather than every page canvas.

Virtualization

`VariableSizeList` mounts pages around the viewport rather than the entire canvas set.

Dimensions

Page aspect ratios inform row sizes.

Scrolling

Loading states cover pages whose rendered canvas is not ready. No fixed mounted-page count or memory benchmark is claimed.

3.2 Zoom rendering

Zoom combines an immediate scaled preview with a replacement render at the requested resolution:

01 Preview the zoom

Apply a CSS `scale()` immediately to provide feedback during the gesture.

02 Render at resolution

The documented delay is 200ms before rendering the PDF at the new resolution.

03 Swap the canvas

Keep the displayed image visible until its replacement is ready. Verify continuity and frame rate on representative documents.

3.3 Image viewer

Implementation `src/components/translations/image-viewer.tsx`

For scanned JPEG or PNG inputs, `react-zoom-pan-pinch` provides image zoom and pan controls.

Transform

The image viewer provides rotation and horizontal/vertical flip controls.

Orientation

The image style sets `imageOrientation: from-image`. Browser and source metadata determine the displayed orientation.

Navigation

Use the viewer's visible zoom and transform controls; inspect the component for supported keyboard bindings.

3.4 Review layout

Implementation `src/components/translations/review-layout.tsx` & `document-viewer.tsx`

`DocumentViewer` selects the viewer for the source format:

01 Detect file type

Checks file extension (and MIME type fallback).

02 Select viewer

Renders `<PdfViewer />` or `<ImageViewer />` dynamically.

03 Preserve layout

Both viewers share the same `isCollapsed` state managed by the parent `ReviewLayout`.

The review layout combines panel resizing, draft state, shortcuts, and narrow-screen behavior:

Panels

`react-resizable-panels` provides source/editor resizing and the stored panel ratio.

Drafts

Autosave is debounced by 5000 milliseconds. A beacon attempt on exit is best-effort; an acknowledged save establishes persistence.

Narrow screens

The review layout uses a `max-width: 1024px` media query to select its mobile layout. The offline preview is a separate evaluation route.

4. Client records

Implementation `src/app/(protected)/app/customers` & `src/actions/services/customers`

Client records let a translation agency group work by its own end-clients.

4.1 Data model

The authenticated account owns the client records and translations within its tenant boundary.

Ownership

`customers.profile_id` references the account's profile; `display_name` identifies the synthetic or user-managed customer record.

Assignment

`translations.customer_id` links a translation to a customer. RLS determines which records the caller may read or change.

4.2 Client operations

Client changes

`add-customer.ts` and `update-customer.ts` perform server-side writes under the user's RLS context. Verify transaction and error handling in those actions before describing them as atomic.

Translation assignment

`assign-customer-modal` links a completed translation to a client record.

5. Realtime synchronization

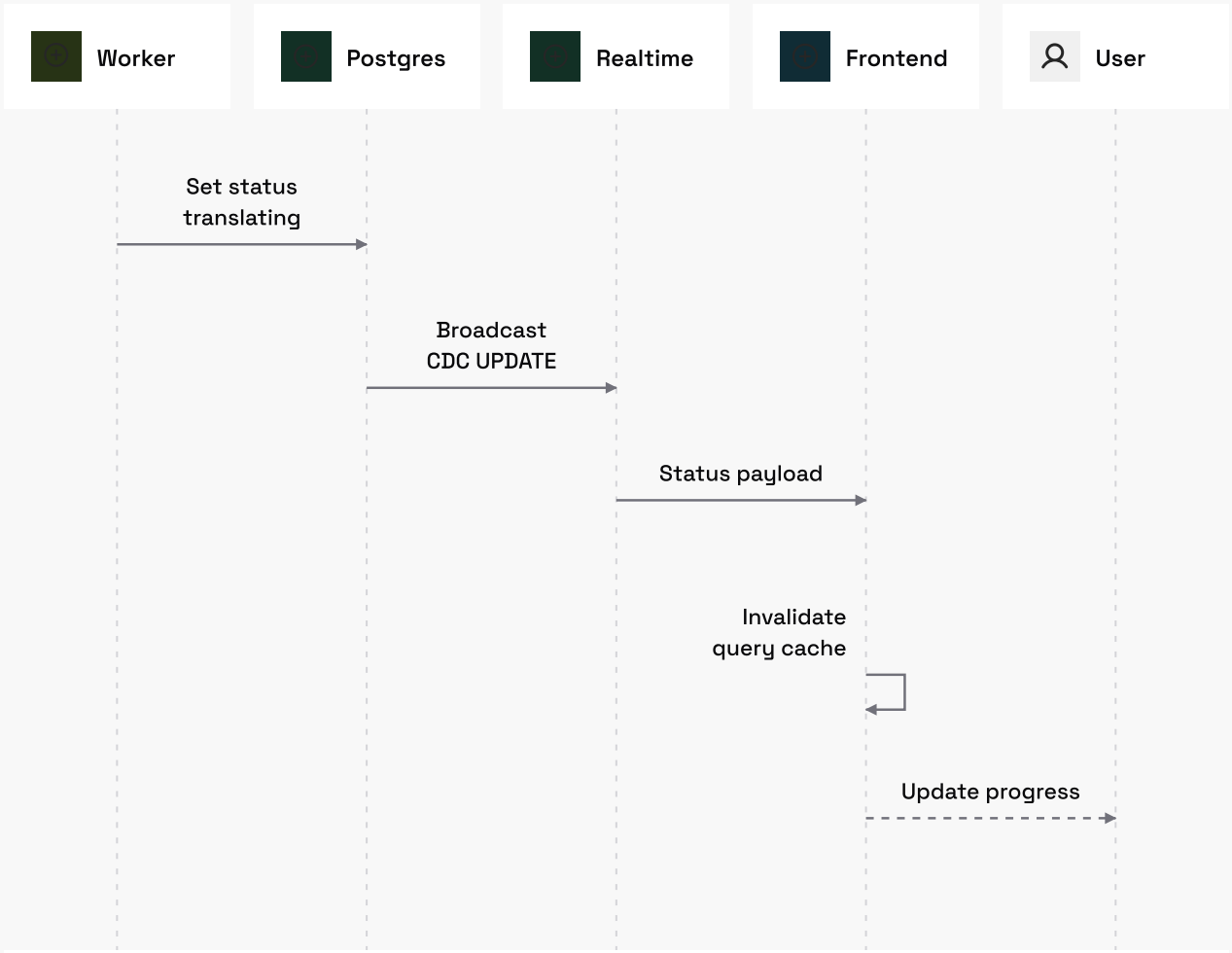
Implementation `src/hooks/use-translation-realtime.ts`

Supabase Realtime carries translation updates to the active workspace. Query refresh or polling can provide a fallback where configured.

5.1 Optimistic state and subscriptions

The shown subscription filters `translations` events by `id = translationId`. Row-level authorization must be enforced by the database; an event filter selects a record and does not itself authorize access.

11 Realtime synchronization



Payload: new.status = ”translating”.
The record filter selects a translation; RLS controls access.

5.2 Subscription hook

The hook manages the subscription channel, record filter, and cleanup.

SRC/HOOKS/USE-TRANSLATION-REALTIME.TS · L21–38 LISTING LINES 1–18

```
export function useTranslationRealtime(
    translationId: string,
    initialData: TranslationRow | null,
): TranslationRow | null {
    const [translation, setTranslation] = useState<TranslationRow | null>(initialData);
    const [supabase] = useState(() => createBrowserClient());

    // Memoize the update handler
    const handleUpdate = useCallback((payload:
RealtimePostgresChangesPayload<TranslationRow>) => {
        if (payload.new && payload.eventType === 'UPDATE') {
            setTranslation(payload.new as TranslationRow);
        }
    }, []);

    useEffect(() => {
        if (!translationId) {
            console.warn('[useTranslationRealtime] Translation ID is missing.');
            return;
```

Event delivery

Realtime events update the active translation without continuous polling. Latency depends on region, connection, and load.

Record selection

The subscription filters events to the translation being viewed.

Authorization

RLS governs which rows the connected user may access. The ID filter is not a substitute for that policy.

6. Workspace resilience

Implementation `src/components/translations/editor-skeleton.tsx`

Long document trees take time to parse and render. Loading and error states keep the workspace understandable during that work.

6.1 Loading skeleton

`EditorSkeleton` provides a toolbar-and-document loading surface while the editor initializes. Its presence does not establish a measured layout-shift result.

SRC/COMPONENTS/TRANSLATIONS/EDITOR-SKELETON.TSX · L10–24 LISTING LINES 1–15

```
01 export function EditorSkeleton({ className }: { className?: string }) {
02     return (
03         <div className={cn('dark:bg-surface-950 flex h-full flex-col bg-white', className)}>
04             {/* Toolbar Skeleton */}
05             <div className="border-surface-2 dark:border-surface-800 sticky top-0 z-10 flex h-14 shrink-0 items-center gap-2 border-b bg-white px-4">
06                 <Skeleton className="h-8 w-24 rounded-lg" />
07                 <div className="h-6 w-px bg-gray-200 dark:bg-gray-700" />
08                 <Skeleton className="h-8 w-8 rounded-lg" />
09                 <Skeleton className="h-8 w-8 rounded-lg" />
10                 <Skeleton className="h-8 w-8 rounded-lg" />
11                 <div className="h-6 w-px bg-gray-200 dark:bg-gray-700" />
12                 <Skeleton className="h-8 w-32 rounded-lg" />
13             </div>
14
15             {/* Content Skeleton */}
```

6.2 Error boundaries

Route error boundaries expose recovery actions when a workspace segment fails. The documented editor fallback offers access to the original document.

7. Workspace technology

COMPONENT	TECH	ROLE
Editor Core	Tiptap with React	Headless content editing engine.
State	`zustand` + `react-query`	Managing editor focus modes and async data fetching.
Styling	Tailwind Merge	Dynamic class construction for custom node views.
Realtime	Supabase Realtime	Websocket subscriptions for status updates.

8. Performance strategies

The workspace uses the following strategies to reduce editor and viewer work:

Editor configuration

`useEditorConfig` memoizes extension configuration.

Table layout

The `kvp-table` class sets fixed table layout.

Source viewer

Dynamic imports separate viewer loading from the surrounding workspace. Profile representative documents to assess performance.

9. Editor decisions

9.1 Why Tiptap and ProseMirror

REQUIREMENT	CHOSEN APPROACH	DESIGN CONSEQUENCE
Custom document structures	ProseMirror schema extensions	Represent signatures, description lists, and other Verto-specific nodes.
Product UI integration	Headless Tiptap editor	Own the toolbar and component styling while using the editor's document model.
Structured interchange	JSON document representation	Keep a typed document tree between processing, editing, and compilation.
Collaboration path	Yjs integration capability	A possible extension path; this comparison does not establish a shipped collaborative feature.

9.2 Why optimistic saving

The documented autosave debounce is 5000ms. Pending changes still need a visible save state and failure recovery.

Local feedback

Update the local editor immediately while the save runs.

Conflict policy

The documented last-write-wins strategy favors a simple single-user workflow. Concurrent tabs or editors can overwrite changes unless the application adds version checks or conflict handling.

Save feedback

Use the `isSaving` state for pending work, and show success only after the server acknowledges the save. Preserve the draft and expose a retry action on failure.

10. User credit consumption

Implementation `src/constants/credits.ts`

The shown credit calculation is intentionally small:

RULE	IMPLEMENTATION	SCOPE
Page calculation	`ceil(pageCount / 1)`	Positive page counts
Minimum	`max(1, calculatedCredits)`	One-credit minimum
Purchase price	Configured unit price and preset totals	Separate from translation credit consumption

PRICING BOUNDARY

`calculateCreditsRequired` uses the one-credit minimum and page rate from `src/constants/credits.ts`. Batch complexity multipliers estimate output size and do not set the user's credit bill.

PART II · THE EXPERIENCE

Governance & operations 05

Administration, permissions, finance, and reporting.

IN THIS SECTION

01 Administrative responsibilities P. 088
02 Privilege boundaries P. 088
03 User governance P. 090
04 Financial operations P. 091
05 Operational reporting P. 091
06 Administration technology P. 093

Modules	`src/app/(protected)/admin/*` · `src/actions/admin/*`

1. Administrative responsibilities

System 5 gives administrators controlled actions for user management, credit adjustments, security recovery, and reporting. Each action needs an explicit permission check, a checked write result, and an audit record; the excerpts do not prove these properties for every action.

2. Privilege boundaries

Implementation `src/utils/supabase/server.ts` (Admin Client) & `src/utils/supabase/middleware.ts`

Ordinary actions operate under the user's RLS context. Cross-tenant administration requires explicit elevation.

2.1 Service-role access

Keep the Supabase `service_role` credential in the authorized server environment.

Authorization

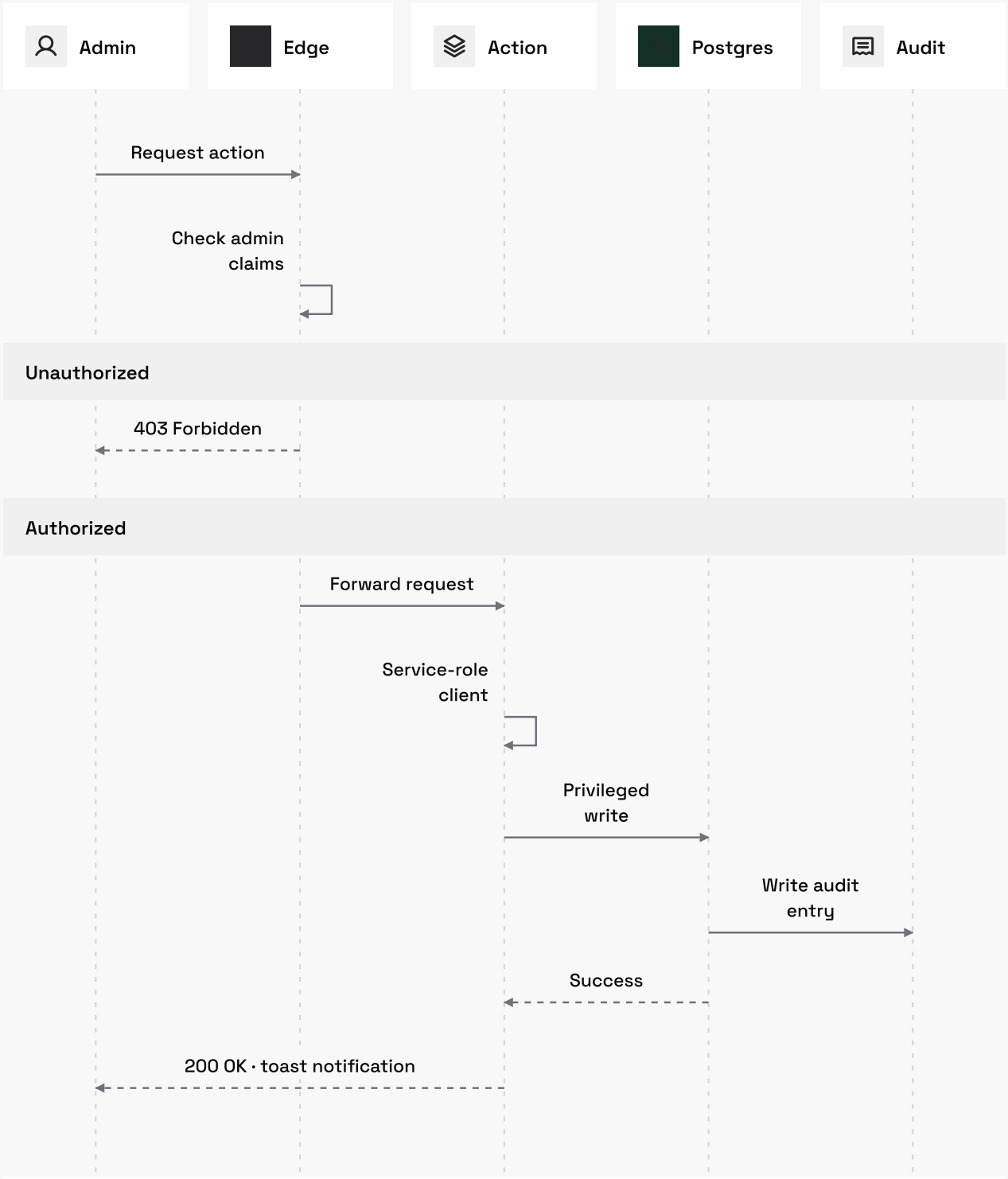
The request and action helpers inspect the authenticated user and administrative role.

Elevation

A server-only service-role client bypasses RLS. Each privileged action must authorize its caller and check write results independently.

2.2 Action execution flow

12 Admin action sequence



The service-role client stays server-side.
Permission, write success and audit outcome are separate checks.

3. User governance

Implementation `src/app/(protected)/admin/users`

User management provides controlled operations for account status and security recovery.

3.1 Management interface

Implementation `src/app/(protected)/admin/users/[id]/user-profile-client.tsx`

A Server Component fetches the administrative view; a Client Component handles interaction.

Query boundary

The administrative profile view composes overview and billing tabs.

Filter scope

The billing section has a scoped `FilteringStoreProvider`. Query keys, account selection and authorization remain separate controls.

3.2 Account suspension

Implementation `src/actions/admin/users/suspend-user.ts`

Suspension preserves account history while restricting access. The action and enforcement path must agree on the suspended state.

01 Update account status

Set `profiles.account_status` to `SUSPENDED` through the authorized administrative action.

02 Enforce the restriction

Middleware and protected operations must consult the account status. Verify the active-session behavior instead of assuming the status write immediately revokes all access.

03 Record the action

Write the actor, reason, and timestamp to the audit record and check the result.

3.3 MFA recovery

Implementation `src/actions/admin/users/manage-mfa.ts`

An administrator can remove a lost-device MFA factor through the provider's management API. Factor removal, session assurance, and active-session revocation are separate concerns; confirm the recovery and re-authentication policy before restoring access.

4. Financial operations

Implementation `src/actions/admin/users/adjust-credits.ts`

Financial interventions create ledger events through the normal accounting path described in System 2.

4.1 Adjustment transactions

Record each balance change as a transaction; do not directly modify `credits.amount`.

Transaction types

The shown adjustment action uses `bonus` for a credit and `penalty` for a debit. These are recognized by `sync_credits`; a generic `adjustment` type is not shown in that trigger.

Audit metadata

Include the administrative actor and reason so the adjustment can be traced.

Balance update

The normal ledger insertion path invokes `sync_credits`. Check the write result before confirming the change to the operator.

5. Operational reporting

Implementation `src/app/(protected)/admin/dashboard`

The dashboard brings processing status and credit usage together for operational review.

5.1 Processing metrics

Reporting

Administrative analytics actions query the available application records for charts and filters.

Interpretation

Record timestamps and language fields can support analysis. The repository supplies no measured processing-latency or language-demand benchmark.

5.2 Credit issuance and usage

Implementation `src/actions/admin/analytics/statistics/get-credits-burn-down-analytics.ts`

Compare issued and consumed credits to understand activity and outstanding balances. Accounting treatment requires a separate policy.

Issued credits

Track credits sold or granted, distinguishing purchases from bonuses.

Consumed credits

Track credits spent on completed translation work.

Outstanding balance

Compare issued, consumed, and remaining credits. These are operational measures; the report does not determine accounting recognition.

6. Administration technology

COMPONENT	TECHNOLOGY	PURPOSE
Data Visualization	`recharts`	Rendering interactive timeseries data.
Data Fetching	**React Server Components**	Secure, server-side data retrieval.
Filter State	`zustand`	Transient state management for data tables.
Design System	**shadcn/ui**	Unified interface components.

Administrative actions require the `ADMIN` role. This is an intended boundary documented in code, not a certification of every deployment or administrative action.

PART II · THE EXPERIENCE

Design engine & brand 06

Components, communication, assets, and experience guardrails.

IN THIS SECTION

01 Product design architecture P. 095
02 Metadata and discovery P. 097
03 Product communication P. 099
04 Error and empty states P. 100
05 Entry routes P. 100
06 Asset management P. 100
07 Component composition P. 101
08 Design and communication technology P. 102

SYSTEM RESPONSIBILITY

The product design layer defines reusable components, semantic tokens, communication patterns, and experience states. This chapter describes Verto's application UI; the handbook itself uses the Yaschet publication system.

1. Product design architecture

`src/app/globals.css` defines the product's semantic CSS tokens using Tailwind CSS and the Harmony palette.

1.1 Component layers

Higher-level components consume shared tokens and primitives. This layer structure gives product teams a common starting point for color, spacing, and interaction.

Foundation

`src/app/globals.css` defines application color tokens and shared rules.

Tokens

Semantic surface, text and state roles adapt to the theme.

Primitives

Radix behavior and CVA variants support the component wrappers.

Composition

Workspace components consume those primitives and tokens.

1.2 Semantic color system

OKLCH represents perceptual lightness, chroma, and hue. Verto's semantic layer maps palette values to roles such as information, success, warning, and error.

Palette foundation

The palette derives from `evilmartians/harmony`. Shared scale positions provide a useful starting relationship across hues; contrast and perceived balance still need checking in each actual component.

Surfaces

Use neutral surfaces for the canvas and structural elements.

Primary ink

Use high-contrast primary text for the core reading hierarchy.

Accent

Reserve the action color for interactive states and primary calls to action.

Feedback

Keep state colors tied to meaning.

- **Success / emerald:** Completed operations.
- **Warning / orange:** Non-blocking alerts.
- **Destructive / red:** Irreversible actions.

Typography

Application text

Cairo is bundled locally under the SIL Open Font License and loaded through Next.js local fonts.

Monospace

Code and utility surfaces use the configured monospace stack.

Publication

The handbook uses Space Grotesk and Space Mono under their accompanying font licenses.

1.3 Motion

Shared motion settings define durations and easing for entry, exit, and continuous effects.

MECHANISM	CONFIGURATION SOURCE	REVIEW BOUNDARY
Enter and exit	Animation rules in `src/app/globals.css`	Component classes select timing.
Component motion	Motion props and local variants	Inspect each interaction's implementation.
Loading	Skeleton and pending-state components	Loading feedback does not prove a performance improvement.

1.4 Interaction feedback

Visible button state, loading indicators and toasts communicate interaction outcomes. Feedback should remain understandable without sound.

State

Use semantic text and icons for pending, success and error outcomes.

Evaluation

The synthetic preview distinguishes tab-local edits from server persistence.

2. Metadata and discovery

Application metadata, manifest icons and document links describe the source-available project.

2.1 Search metadata

`src/library/app.ts` centralizes application metadata:

Site constants

`src/library/app.ts` provides the application title, description and origin.

Social preview

The repository includes a synthetic 1280 × 640 social image and an application OG route.

Indexing

Review `robots.ts` and `sitemap.ts` for the routes the application exposes.

2.2 Discovery

Sitemap

`src/app/sitemap.ts` declares the public routes.

Manifest

`src/app/manifest.ts` declares the app's icon and display metadata. A manifest alone does not establish offline PWA support.

2.3 Privacy boundary

The local preview loads synthetic data and does not require visitor analytics or production accounts.

Local preview

The offline runner blocks outbound Node connections and uses inert configuration.

Browser evidence

The reviewed preview requests local assets. Connected application routes have separate service requirements.

3. Product communication

Toasts, transactional email, and authentication email share state terminology and brand conventions while serving different delivery contexts.

3.1 Toast notifications

`src/components/ui/sonner.tsx` adapts Sonner to the product's notification patterns.

Toast layout

Use compact floating cards and the product's `Surface-2` token.

State icons

Pair semantic feedback with the appropriate Phosphor mark, such as a check for success or a warning symbol for an error.

Theme support

Adapt notification surfaces and ink to the current light or dark theme.

3.2 Transactional email

Templates in `src/emails` use React Email to carry the product identity into transactional messages.

Templates

`src/emails/translation-complete.tsx` and `translation-failed.tsx` define transactional content.

Delivery

The Resend adapter requires your configured sender. Test links, permissions and email clients before connected use.

3.3 Authentication email

Supabase Auth sends authentication email under the configuration of your own project. This repository does not provision a sender or a provider-hosted email template.

Identity

Keep the application origin and callback allowlist aligned.

Delivery

Verify account messages and recovery links through your configured authentication provider.

4. Error and empty states

Error, loading, and empty states explain what happened and give the reader a useful next action.

Errors

`src/app/error.tsx` displays failure details and recovery/navigation controls.

Loading

`src/app/loading.tsx` renders the product mark inside `PrivacyBlurWrapper`.

Missing route

`src/app/not-found.tsx` provides the application's missing-page interface.

5. Entry routes

The root page introduces the source-available application. The synthetic editor lives at `/preview`; authenticated workflows live under protected routes.

Project entry

The root page links to evaluation, documentation and the case study.

Evaluation

`/preview` runs the real editor with English, Arabic and Japanese synthetic documents.

6. Asset management

Static brand assets live in `public`, grouped by purpose and variant.

Brand

`public/brand/` contains Verto logo and mark variants.

Publication media

`.github/assets/` holds synthetic light/dark workspace captures and the social preview.

Fonts

`public/fonts/` contains Cairo and its license; fixture PDFs embed licensed font subsets.

7. Component composition

The UI library in `src/components/ui` composes low-level primitives into reusable product components.

7.1 Composition pipeline

The component library combines headless behavior with shared visual styling:

Radix behavior

Handles state (`data-state="open"`), accessibility, and keyboard navigation.

CVA variants

Maps props (e.g., `size="sm"`) to Tailwind classes.

Theme tokens

Resolves semantic tokens (e.g., `bg-primary`) to the current theme's OKLCH values.

Motion

Applies motion to state changes (e.g., `layoutId` transitions).

Composed visual components build on those primitives:

Document controls

Toolbar, table menus and link controls compose the shared UI primitives.

Source viewer

PDF and image viewers keep the uploaded source available during review.

8. Design and communication technology

AREA	TECHNOLOGY	PURPOSE
Styling	Tailwind CSS and OKLCH	Semantic light/dark surfaces
Primitives	Radix UI and CVA	State, variants and keyboard behavior
Motion	Motion libraries	Component transitions
Email	React Email and Resend	Configured transactional delivery
Feedback	Sonner and Phosphor	Visible status and controls
Typography	Cairo	Local application font

PART III · INFRASTRUCTURE

Infrastructure & orchestration

07

Hosting, networking, security, and deployment.

IN THIS SECTION

01 Hosting, state, and compute P. 104

02 Network resilience P. 105

03 Environment and security P. 107

04 Deployment workflow P. 108

05 Infrastructure register P. 109

SYSTEM RESPONSIBILITY

Infrastructure separates application requests, persistent state, and background processing. This chapter covers their network limits, credentials, and deployment boundaries.

1. Hosting, state, and compute

Application hosting handles Next.js requests; Supabase holds application state and Trigger.dev executes durable processing. The source-available repository does not provision a deployment.

1.1 Application hosting

Application host

Next.js handles request-bound application work.

Processing

Translation and native document generation execute in background tasks.

Deployment limits

Hosting plans and invocation limits are external configuration. Check the selected host rather than treating this book as a deployment contract.

1.2 Persistent state

Role

Persist application data and synchronize state.

Database boundary

Use transactions and row-level security for tenant data.

Responsibilities

Postgres: credits, users, and documents.

Storage: source PDFs and processed assets.

Realtime: translation state over WebSocket connections.

1.3 Background execution

Worker

Trigger.dev executes the task graph and its MuPDF system dependency.

Budget

The translation parent and vision task request up to 86400 seconds. The vision task selects the `medium-1x` preset.

Progress

Database updates and task metadata expose processing state; neither implies a completion deadline.

2. Network resilience

Long-running inference needs explicit timeout, retry, and checkpoint policies. Client configuration cannot override an upstream proxy or provider's connection limits.

2.1 HTTP client timeouts

Undici separates connection establishment, response-header, response-body, and idle-connection timeouts. Tune each for the expected workload and hosting path; a long idle timeout does not keep an active request alive indefinitely.

The source injects a custom `undici.Dispatcher` with explicit timeout values:

SRC/UTILS/AI/OPENAI-RESPONSES-CLIENT.TS · L29–38 LISTING LINES 1–10

```
    keepAliveTimeout: 30000, // Keep connection active for 30s
    keepAliveMaxTimeout: 7200000, // Max keepalive timeout 2 hours
    bodyTimeout: 7200000, // 2 hours body timeout
    headersTimeout: 7200000, // 2 hours headers timeout
});

// Initialize OpenAI client
const openai = new OpenAI({
    apiKey: env.OPEN_AI_API_KEY,
    timeout: 7200 * 1000, // 2 hours explicit timeout (globally applied)
```

`keepAliveMaxTimeout` bounds idle reuse; header and body settings bound different request phases. These values cannot force an upstream connection to remain open.

2.2 Error taxonomy

Implementation `classify-error.ts`

Classify failures before choosing retry, recovery, or a terminal error:

CLASS	POLICY SOURCE	RECOVERY BOUNDARY
Transient provider/network error	`trigger/errors/classify-error.ts`	Retry within configured limits.
Permanent input or authentication error	Classification and task failure handler	Inspect input, credentials and terminal state.
Truncated structured output	JSON recovery and batch sizing	Reduce batches, validate the result and retain review warnings.

2.3 JSON repair

The parser attempts heuristic JSON repair after malformed or truncated output. Validate the result and reject missing required content.

01 Stack balancer

Detects unclosed `[{` and synthetically closes them.

02 Node scraper

If JSON is severely malformed, it regex-scrapes valid `ProseMirror` nodes to salvage partial work.

3. Environment and security

3.1 Environment validation

Implementation `env.ts`

`t3-oss/env-nextjs` validates environment values against a schema. Missing or invalid required values should fail configuration validation before the application serves requests.

Service-role credential

Keep `SUPABASE_SERVICE_ROLE_KEY` in the server environment. Validate it with the deployed provider configuration; a fixed string prefix is not a complete credential check.

Worker credential

Keep `TRIGGER_SECRET_KEY` in the worker's secret configuration. Validate the configured environment and permissions.

Application URL

Set `NEXT_PUBLIC_APP_URL` to the canonical application origin and use an explicit redirect allowlist.

3.2 Service-role isolation

Browser client

Authenticate the user and access only data permitted by RLS. Never distribute the service-role credential to the browser.

User server actions

Perform authorized operations in the user's context and check results.

Administrative actions

Use `supabaseAdmin` only after an explicit administrative authorization check; its elevated access bypasses RLS.

4. Deployment workflow

The frontend and background worker have separate deployment steps. Coordinate compatible versions and environment values across both services.

4.1 Frontend deployment

Verify

Run repository checks and the offline build before preparing a deployment.

Configure

Set your host's environment from the typed schema and verify the intended repository and branch.

Release

Deploy through your chosen host's workflow. Repository release automation does not deploy the application.

4.2 Worker deployment

Deploy the background worker separately from the application host under your own service configuration and authorization.

Configure

Set `TRIGGER_PROJECT_REF` and the worker's validated service settings for your account.

Deploy

Use the Trigger.dev CLI and the checked-in `trigger.config.ts` under your own authorization.

Verify

Confirm task versions, runtime dependencies and environment in your Trigger.dev project before accepting work.

5. Infrastructure register

PROVIDER BOUNDARY	RESPONSIBILITY	CONFIGURATION
Application host	Next.js requests and static assets	Choose your host and runtime.
Supabase	Auth, Postgres, storage and realtime	Own project and RLS policies
Trigger.dev	Background task execution	Project, machine and retry settings
OpenAI	Translation and review adapters	Models, usage and provider quotas
Google	Docs and Drive generation	Service-account access and API quotas

REFERENCE · APPENDIX

Technical dependencies.

The platforms and services that support the Verto ecosystem.

VERTO
REF
SERVICE REGISTER

The service register identifies provider boundaries and the configuration each requires.

PROVIDER BOUNDARY	RESPONSIBILITY	CONFIGURATION
Application host	Next.js application requests	Choose and verify your host.
Supabase	Postgres, Auth, storage and realtime	Own project, roles and policies
Trigger.dev	Durable processing tasks	Worker runtime and environment
OpenAI	Language and vision adapters	Models, usage and quotas
Stripe	Checkout, portal and webhooks	Own account and event configuration
Google	Docs and Drive generation	Service-account access
Resend	Transactional delivery	Sender and domain configuration



Verto

Engineering Handbook

AUTHOR

Yassine Chettouch

Product Engineer

PUBLICATION

January 2026

Technical handover

PROJECT CASE STUDY

yaschet.com/case-studies/verto